NetApp AI 控制平台

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/ai/aicp_introduction.html on March 19, 2024. Always check docs.netapp.com for the latest.

目录

NetApp AI 控制平台 1
TR-4798： NetApp AI 控制平台 1
概念和组件 2
硬件和软件要求 9
Kubernetes 部署 10
NetApp Trident 部署和配置 11
Kubeflow 部署 17
Kubeflow 操作和任务示例 26
Apache Airflow 部署 34
Apache 气流工作流示例 37
Trident 操作示例 37
ONTAP AI 部署的高性能作业示例 40
性能测试 52
结论 52

NetApp AI 控制平台

TR-4798： NetApp AI 控制平台

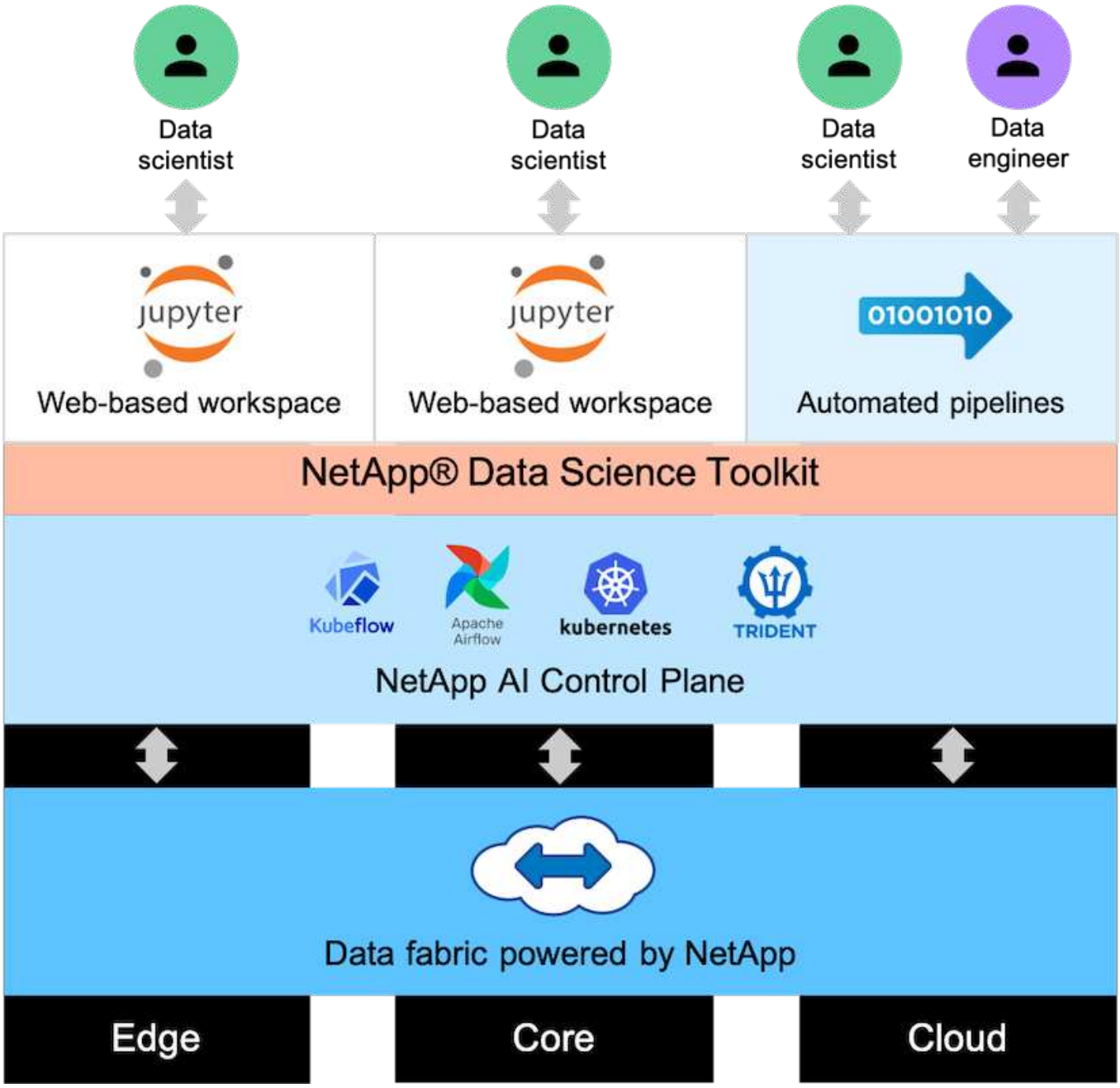
NetApp 公司 Mike Oglesby

各行各业各种规模的企业和组织都在转向人工智能（AI），机器学习（ML）和深度学习（DL），以解决实际问题，提供创新产品和服务，并在竞争日益激烈的市场中占据优势。随着企业越来越多地使用 AI， ML 和 DL，他们面临着许多挑战，包括工作负载可扩展性和数据可用性。本文档演示了如何使用 NetApp AI 控制平台来应对这些挑战，NetApp AI 控制平台是一种将 NetApp 数据管理功能与常见开源工具和框架配对的解决方案。

此报告介绍如何快速克隆数据命名空间。此外，还将向您展示如何在站点和区域之间无缝复制数据，以创建统一的统一 AI/ML/DL 数据管道。此外，它还会指导您完成 AI， ML 和 DL 培训工作流的定义和实施，这些工作流可近乎即时地创建数据和模型基线，以实现可追溯性和版本控制。使用此解决方案，您可以跟踪每个模型训练返回到用于训练和 / 或验证模型的确切数据集。最后，本文档将向您介绍如何快速配置 Jupyter 笔记本电脑工作空间，以便访问海量数据集。

注意：对于涉及大量需要共享访问同一数据集的 GPU 服务器的 HPC 模式大规模分布式培训，或者如果您需要 / 更喜欢并行文件系统，请查看 "TR-4890"。本技术报告介绍了如何包括 "NetApp 完全支持的并行文件系统解决方案 BeeGFS" 作为 NetApp AI 控制平台的一部分。此解决方案可从少数 NVIDIA DGX A100 系统扩展到全闪满的 140 节点 SuperPOD 。

NetApp AI 控制平台面向数据科学家和数据工程师，因此只需极少的 NetApp 或 NetApp ONTAP ® 专业知识即可。借助此解决方案，可以使用简单熟悉的工具和界面来执行数据管理功能。如果您的环境中已有 NetApp 存储，您可以立即测试运行 NetApp AI Control 平台。如果您要测试解决方案驱动器，但尚未安装 NetApp 存储，请访问 "cloud.netapp.com"，您只需几分钟即可使用基于云的 NetApp 存储解决方案启动并运行。下图显示了解决方案的可视化视图。



概念和组件

人工智能

AI 是一门计算机科学学科，其中计算机经过训练，可以模拟人类思维的认知功能。AI 开发人员培训计算机，以便以类似于甚至优于人类的方式学习和解决问题。深度学习和机器学习是 AI 的子领域。企业越来越多地采用 AI ， ML 和 DL 来满足其关键业务需求。以下是一些示例：

- 分析大量数据以挖掘以前未知的业务洞察力
- 使用自然语言处理直接与客户互动
- 自动化执行各种业务流程和功能

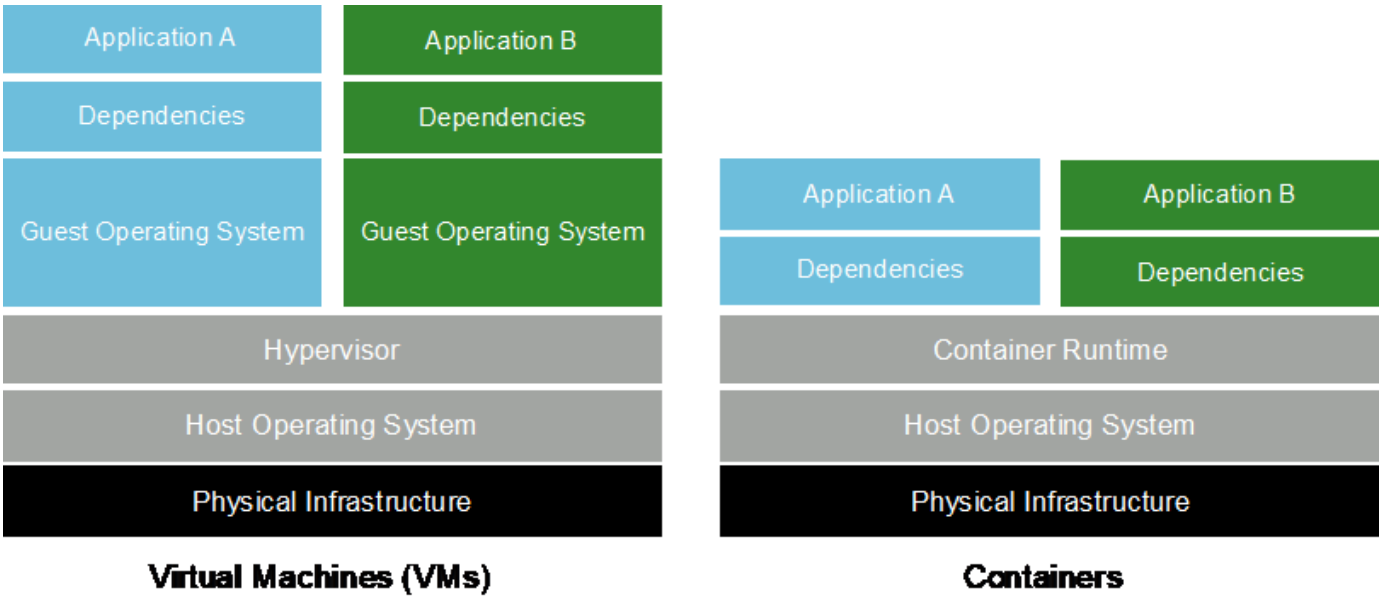
现代 AI 训练和推理工作负载需要大规模并行计算功能。因此， GPU 越来越多地用于执行 AI 操作，因为 GPU

的并行处理功能远远优于通用 CPU 。

容器

容器是在共享主机操作系统内核上运行的隔离用户空间实例。容器的采用率正在快速增长。容器可提供许多与虚拟机（ VM ）相同的应用程序沙盒优势。但是，由于虚拟机所依赖的虚拟机管理程序和子操作系统层已被消除，因此容器的重量要轻得多。下图展示了虚拟机与容器的可视化情况。

此外，还可以通过容器直接将应用程序依赖关系，运行时间等内容高效地打包到应用程序中。最常用的容器打包格式是 Docker 容器。已采用 Docker 容器格式进行容器化的应用程序可以在可以运行 Docker 容器的任何计算机上执行。即使计算机上不存在应用程序的依赖关系，也是如此，因为所有依赖关系都打包在容器中。有关详细信息，请访问 "Docker 网站"。



Kubernetes

Kubernetes 是一款开源分布式容器编排平台，最初由 Google 设计，现在由 Cloud 原生计算基金会（ CNCF ）维护。Kubernetes 可以为容器化应用程序实现部署，管理和扩展功能的自动化。近年来， Kubernetes 已成为主导容器业务流程平台。虽然支持其他容器打包格式和运行时间，但 Kubernetes 最常用作 Docker 容器的业务流程系统。有关详细信息，请访问 "Kubernetes 网站"。

NetApp Trident

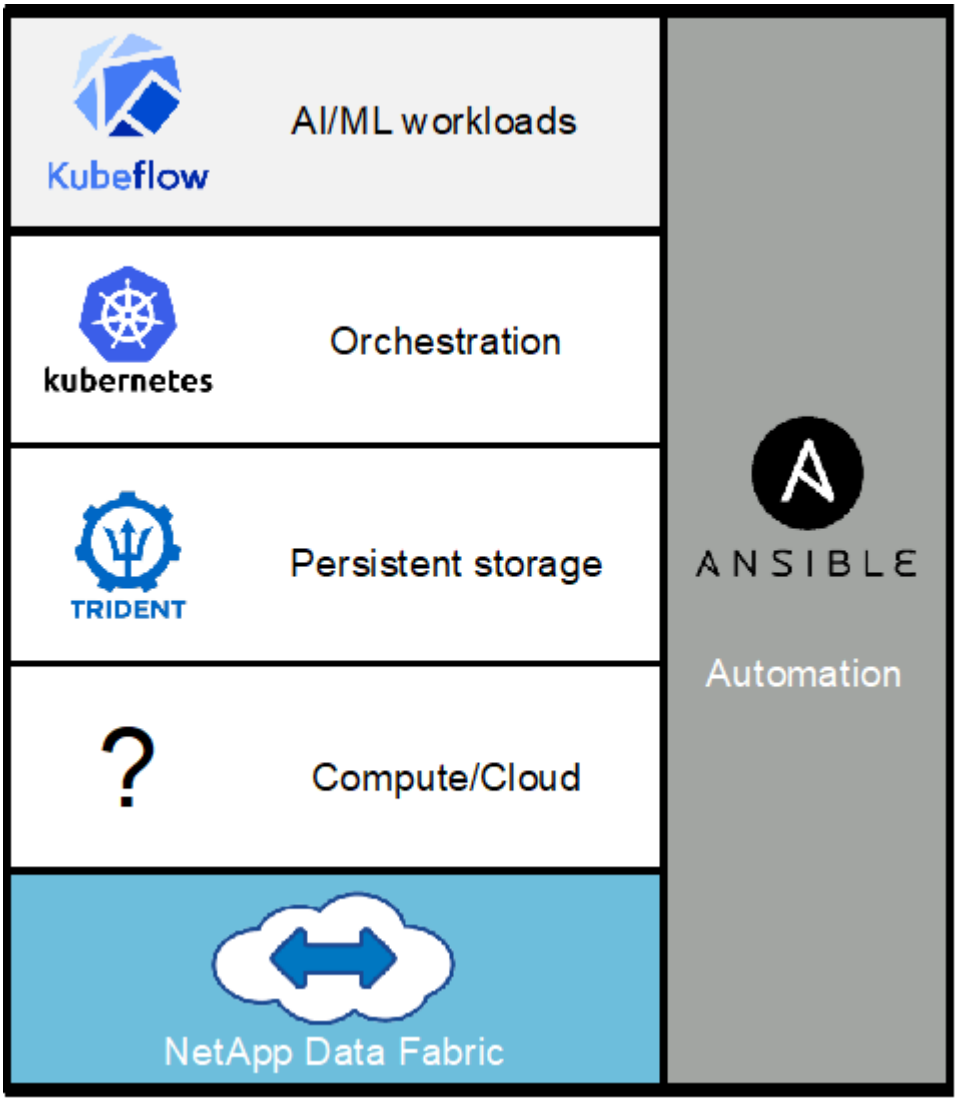
Trident 是一款由 NetApp 开发和维护的开源存储编排程序，可大大简化 Kubernetes 工作负载的永久性存储的创建，管理和使用。Trident 本身是 Kubernetes 本机应用程序，直接在 Kubernetes 集群中运行。借助 Trident ，Kubernetes 用户（开发人员，数据科学家， Kubernetes 管理员等）可以采用他们已熟悉的标准 Kubernetes 格式创建，管理永久性存储卷并与其交互。同时，他们还可以利用 NetApp 的高级数据管理功能以及由 NetApp 技术提供支持的数据网络结构。Trident 可将持久存储的复杂性抽象化，并使其易于使用。有关详细信息，请访问 "Trident 网站"。

NVIDIA DeepOps

DeepOps 是 NVIDIA 的一个开源项目，通过使用 Ansible ，可以根据最佳实践自动部署 GPU 服务器集群。DeepOps 采用模块化设计，可用于执行各种部署任务。在本文档及其所述的验证练习中， DeepOps 用于部署一个由 GPU 服务器辅助节点组成的 Kubernetes 集群。有关详细信息，请访问 "DeepOps 网站"。

Kubeflow

Kubeflow 是一款适用于 Kubernetes 的开源 AI 和 ML 工具包，最初由 Google 开发。通过 Kubeflow 项目，可以在 Kubernetes 上轻松，便携且可扩展地部署 AI 和 ML 工作流。Kubeflow 将 Kubernetes 的错综复杂之处抽象出来，让数据科学家能够专注于他们最了解的—数据科学。有关可视化效果，请参见下图。随着企业 IT 部门越来越多地在 Kubernetes 上实现标准化， Kubeflow 的吸引力也越来越大。有关详细信息，请访问 "Kubeflow 网站"。



KubeFlow 管道

Kubeflow 管道是 Kubeflow 的一个关键组件。Kubeflow 管道是一个平台和标准，用于定义和部署可移植且可扩展的 AI 和 ML 工作流。有关详细信息，请参见 "Kubeflow 官方文档"。

Jupyter 笔记本电脑服务器

Jupyter 笔记本电脑服务器是一款开源 Web 应用程序，数据科学家可以利用它创建类似于维基的文档，这些文档称为 Jupyter 笔记本电脑，其中包含实时代码以及描述性测试。Jupyter 笔记本电脑在 AI 和 ML 社区中广泛使用，可用于记录，存储和共享 AI 和 ML 项目。Kubeflow 可简化 Kubernetes 上 Jupyter 笔记本电脑服务器的配置和部署。有关 Jupyter 笔记本电脑的详细信息，请访问 "Jupyter 网站"。有关 Kubeflow 环境中 Jupyter 笔记本电脑的详细信息，请参见 "Kubeflow 官方文档"。

Apache 气流

Apache Airflow 是一个开源工作流管理平台，支持对复杂的企业工作流进行编程创作，计划和监控。它通常用于自动执行 ETL 和数据管道工作流，但不限于这些类型的工作流。气流项目由 Airbnb 发起，但此后在业内备受欢迎，现在由 Apache 软件基金会赞助。气流采用 Python 编写，气流工作流通过 Python 脚本创建，气流是按照 " 配置即代码 " 原则设计的。 现在，许多企业级气流用户都在 Kubernetes 上运行 Airflow 。

定向循环图（ **DAG** ）

在气流模式下，工作流称为定向环比图（ DAG ）。DAG 由按顺序，并行或两者的组合执行的任务组成，具体取决于 DAG 定义。气流计划程序会根据 DAG 定义中指定的任务级别依赖关系对一组工作负载执行各个任务。DAG 是通过 Python 脚本定义和创建的。

NetApp ONTAP 9.

NetApp ONTAP 9 是 NetApp 推出的最新一代存储管理软件，可帮助像您这样的企业打造现代化的基础架构并过渡到云就绪数据中心。借助行业领先的数据管理功能，无论数据位于何处， ONTAP 都可以通过一组工具来管理和保护数据。您还可以将数据自由移动到任何需要的位置：边缘，核心或云。ONTAP 9 包含许多功能，可简化数据管理，加快和保护关键数据，并在混合云架构中打造适应未来需求的基础架构。

简化数据管理

数据管理对于企业 IT 运营至关重要，这样您就可以为应用程序和数据集使用适当的资源。ONTAP 包括以下功能，可简化运营并降低总运营成本：

- * 实时数据缩减和扩展的重复数据删除。 * 数据缩减可减少存储块中浪费的空间，重复数据删除可显著提高有效容量。
- * 最低，最高和自适应服务质量（ QoS ）。 * 细粒度 QoS 控制有助于在高度共享的环境中保持关键应用程序的性能水平。
- * ONTAP FabricPool 。 * 此功能可将冷数据自动分层到公有和私有云存储选项，包括 Amazon Web Services （ AWS ）， Azure 和 NetApp StorageGRID 基于对象的存储。

加速和保护数据

ONTAP 可提供卓越的性能和数据保护，并通过以下功能扩展这些功能：

- * 高性能和低延迟。 * ONTAP 以尽可能低的延迟提供最高的吞吐量。
- * NetApp ONTAP FlexGroup 技术 * 。 FlexGroup 卷是一种高性能数据容器，可线性扩展到高达 20 PB 和 4000 亿个文件，从而提供一个可简化数据管理的命名空间。
- * 数据保护。 * ONTAP 提供内置数据保护功能，并在所有平台之间进行通用管理。
- * NetApp 卷加密。 * ONTAP 提供原生卷级加密，并支持板载和外部密钥管理。

Future-Proof 基础架构

ONTAP 9 可帮助您满足不断变化的苛刻业务需求：

- * 无缝扩展和无中断运行。 * ONTAP 支持向现有控制器和横向扩展集群无中断添加容量。您可以升级到 NVMe 和 32 Gb FC 等最新技术，而无需进行昂贵的数据迁移或中断。
- * 云连接。 * ONTAP 是云连接最广泛的存储管理软件之一，可在所有公有云中选择软件定义的存储（

ONTAP Select ）和云原生实例（ NetApp Cloud Volumes Service ）。

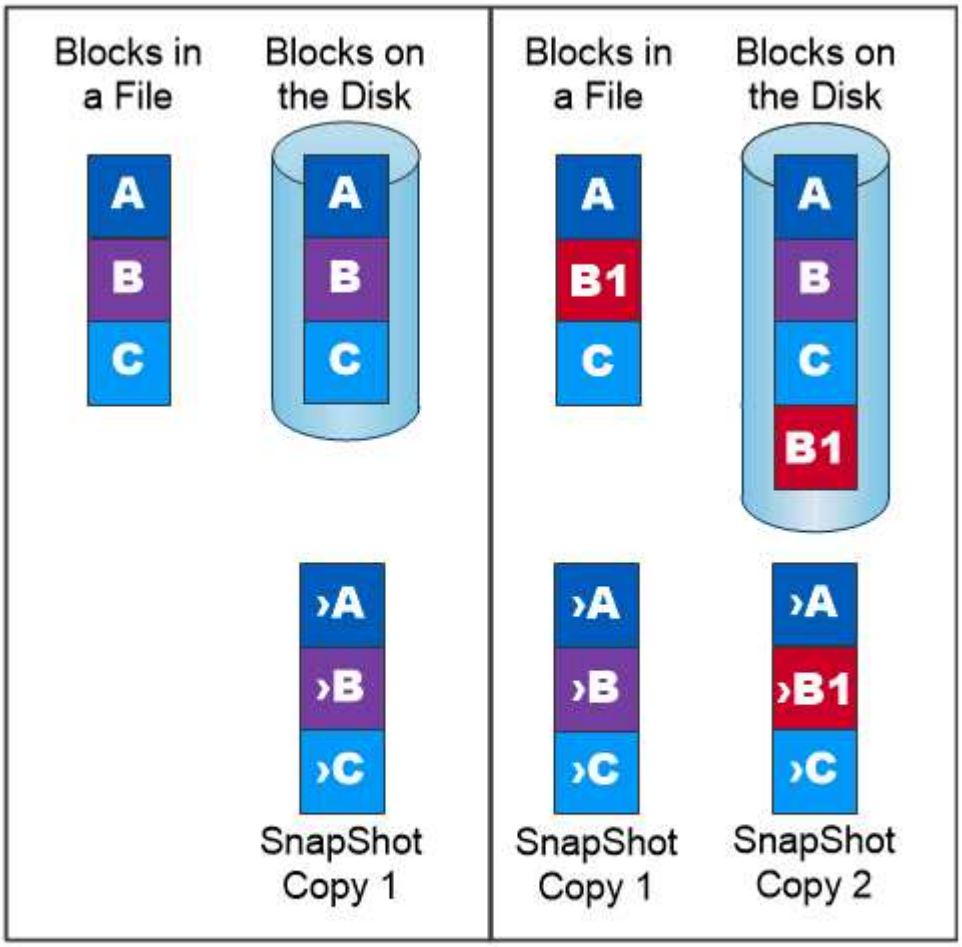
- * 与新兴应用程序集成。 * 通过使用支持现有企业级应用程序的相同基础架构， ONTAP 可为 OpenStack ，Hadoop 和 MongoDB 等下一代平台和应用程序提供企业级数据服务。

NetApp Snapshot 副本

NetApp Snapshot 副本是卷的只读时间点映像。该映像占用的存储空间极少，并且性能开销极低，因为它仅记录自创建上次 Snapshot 副本以来创建的文件所做的更改，如下图所示。

Snapshot 副本的效率归功于核心 ONTAP 存储虚拟化技术—任意位置写入文件布局（ Write Anywhere File Layout ， WAFL ）。与数据库一样， WAFL 使用元数据指向磁盘上的实际数据块。但是，与数据库不同，WAFL 不会覆盖现有块。它会将更新后的数据写入新块并更改元数据。这是因为 ONTAP 在创建 Snapshot 副本时引用元数据，而不是复制数据块，因此 Snapshot 副本的效率非常高。这样做可以避免其他系统在查找要复制的块时花费寻道时间，并避免创建副本本身的成本。

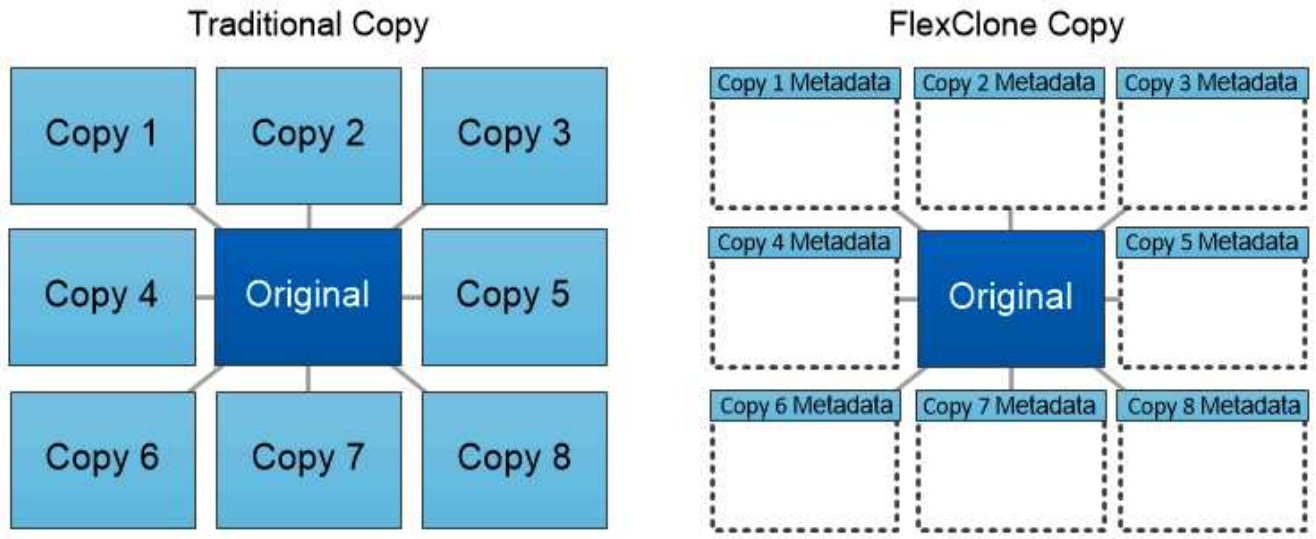
您可以使用 Snapshot 副本恢复单个文件或 LUN ，或者还原卷的整个内容。ONTAP 会将 Snapshot 副本中的指针信息与磁盘上的数据进行比较，以重建缺少或损坏的对象，而不会造成停机或高昂的性能成本。



A Snapshot copy records only changes to the active file system since the last Snapshot copy.

NetApp FlexClone 技术

NetApp FlexClone 技术会引用 Snapshot 元数据来创建卷的可写时间点副本。副本与其父级共享数据块，在将更改写入副本之前，除了元数据所需的存储外，不会占用任何其他存储，如下图所示。传统副本可能需要几分钟甚至几小时才能创建，而 FlexClone 软件可以让您几乎即时复制最大的数据集。因此，如果您需要相同数据集的多个副本（例如，开发工作空间）或数据集的临时副本（针对生产数据集测试应用程序），则这种情况是理想之选。

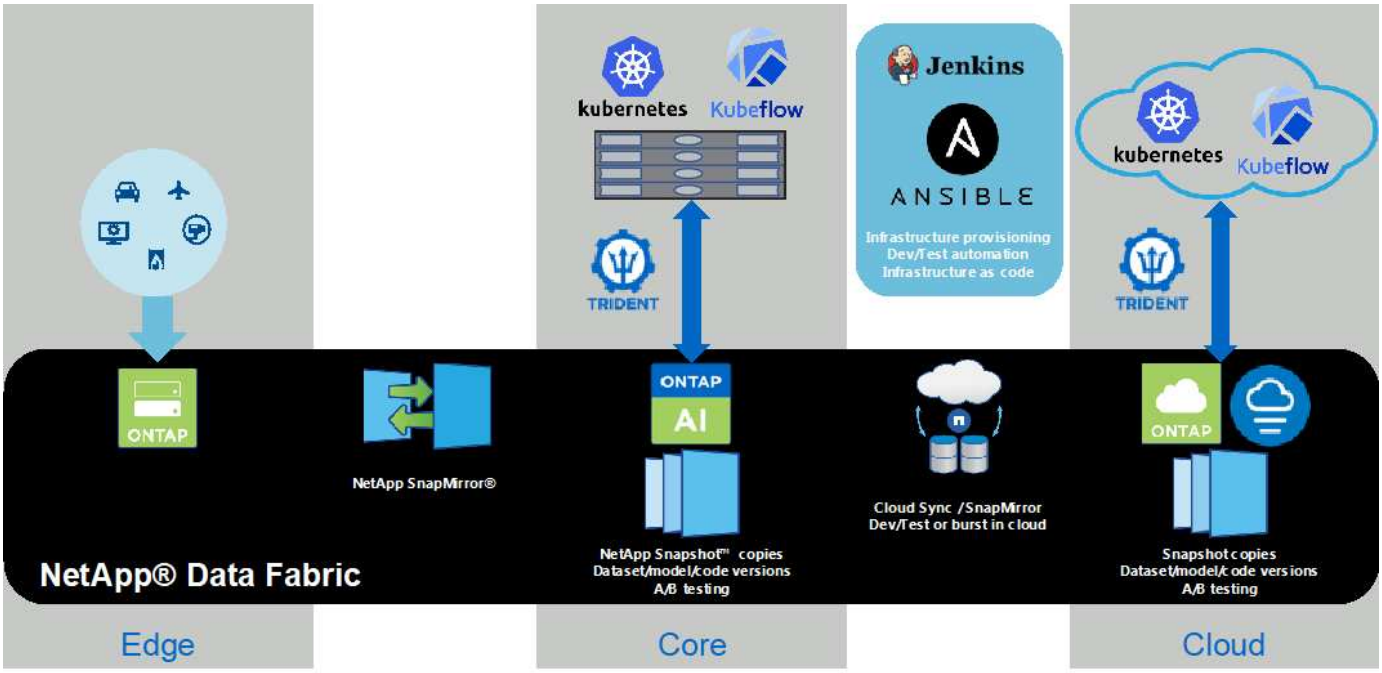


FlexClone copies share data blocks with their parents, consuming no storage except what is required for metadata.

NetApp SnapMirror 数据复制技术

NetApp SnapMirror 软件是一款经济高效且易于使用的统一复制解决方案，可跨数据网络结构实现。它可以通过 LAN 或 WAN 高速复制数据。它可以为各种类型的应用程序提供高数据可用性和快速数据复制，包括虚拟和传统环境中的业务关键型应用程序。在将数据复制到一个或多个 NetApp 存储系统并持续更新二级数据时，您的数据将保持最新，并可随时使用。不需要外部复制服务器。有关利用 SnapMirror 技术的架构示例，请参见下图。

SnapMirror 软件通过仅通过网络发送更改的块来利用 NetApp ONTAP 的存储效率。SnapMirror 软件还可使用内置网络压缩来加快数据传输速度，并将网络带宽利用率降低多达 70% 。借助 SnapMirror 技术，您可以利用一个精简复制数据流创建一个存储库，同时维护活动镜像和先前的时间点副本，从而将网络流量减少多达 50% 。



NetApp BlueXP复制和同步

BlueXP复制和同步是一项NetApp服务、用于快速安全地同步数据。无论您是需要在内部NFS还是SMB文件共享、NetApp StorageGRID、NetApp ONTAP S3、NetApp Cloud Volumes Service、Azure NetApp Files、AWS S3、AWS EFS、Azure Blb、 Google Cloud Storage或IBM Cloud Object Storage、BlueXP Copy and Sync可将文件快速安全地移动到您需要的位置。

数据传输完成后，即可在源和目标上完全使用。BlueXP复制和同步功能可以在触发更新时按需同步数据、也可以根据预定义的计划持续同步数据。不管怎样、BlueXP复制和同步功能只会移动增量、因此可以最大限度地减少数据复制所需的时间和资金。

BlueXP Copy and Sync是一款软件即服务(SaaS)工具、设置和使用极其简单。由BlueXP复制和同步触发的数据传输由数据代理执行。BlueXP复制和同步数据代理可以部署在AWS、Azure、Google Cloud Platform或内部环境中。

NetApp XCP

NetApp XCP 是一款基于客户端的软件，可用于任意到 NetApp 以及 NetApp 到 NetApp 的数据迁移和文件系统洞察。XCP 旨在通过利用所有可用系统资源来处理大容量数据集和高性能迁移来实现扩展和最大性能。XCP 可通过生成报告的选项帮助您全面了解文件系统。

NetApp XCP 可通过一个软件包提供，该软件包支持 NFS 和 SMB 协议。XCP 包括一个适用于 NFS 数据集的 Linux 二进制文件和一个适用于 SMB 数据集的 Windows 可执行文件。

NetApp XCP 文件分析是一款基于主机的软件，可检测文件共享，对文件系统运行扫描并提供用于文件分析的信息板。XCP 文件分析与 NetApp 和非 NetApp 系统兼容，并可在 Linux 或 Windows 主机上运行，以便为 NFS 和 SMB 导出的文件系统提供分析。

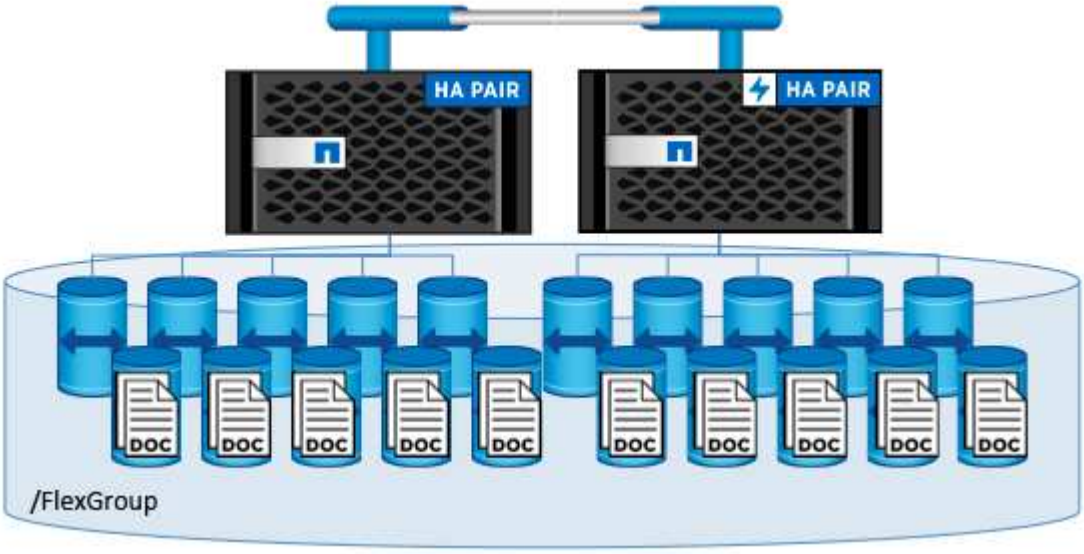
NetApp ONTAP FlexGroup 卷

培训数据集可以是一组可能包含数十亿个文件的集合。文件可以包括文本，音频，视频以及其他形式的非结构化数据，这些数据必须进行存储和处理才能并行读取。存储系统必须存储大量小文件，并且必须并行读取这些文件

，以便执行顺序和随机 I/O

FlexGroup 卷是一个包含多个成分卷的命名空间，如下图所示。从存储管理员的角度来看， FlexGroup 卷是一个受管卷，其作用类似于 NetApp FlexVol 卷。FlexGroup 卷中的文件将分配给各个成员卷，并且不会在卷或节点之间进行条带化。它们支持以下功能：

- FlexGroup 卷可为高元数据工作负载提供多 PB 的容量和可预测的低延迟。
- 它们在同一命名空间中最多支持 4000 亿个文件。
- 它们支持在 CPU ，节点，聚合和成分卷之间的 NAS 工作负载中执行并行操作 FlexVol 。



硬件和软件要求

NetApp AI 控制平面解决方案不依赖于此特定硬件。解决方案与 Trident 支持的任何 NetApp 物理存储设备，软件定义的实例或云服务兼容。例如， NetApp AFF 存储系统，Azure NetApp Files ， NetApp Cloud Volumes Service ， NetApp ONTAP Select 软件定义的存储实例或 NetApp Cloud Volumes ONTAP 实例。此外，只要 Kubernetes 和 NetApp Trident 支持所使用的 Kubernetes 版本，即可在任何 Kubernetes 集群上实施解决方案。有关 Kubeflow 支持的 Kubernetes 版本列表，请参见 "Kubeflow 官方文档"。有关 Trident 支持的 Kubernetes 版本列表，请参见 "Trident 文档"。有关用于验证解决方案的环境的详细信息，请参见下表。

基础架构组件	数量	详细信息	操作系统
部署跳转主机	1.	虚拟机	Ubuntu 20.04.2 LTS
Kubernetes 主节点	1.	虚拟机	Ubuntu 20.04.2 LTS
Kubernetes 工作节点	2.	虚拟机	Ubuntu 20.04.2 LTS
Kubernetes GPU 工作节点	2.	NVIDIA DGX-1 （裸机）	NVIDIA DGX OS 4.0.5 （基于 Ubuntu 18.04.2 LTS ）

基础架构组件	数量	详细信息	操作系统
存储	1 个 HA 对	NetApp AFF A220	NetApp ONTAP 9.7 P6

软件组件	**version**
Apache 气流	2.0.1
Apache 气流 Helm 图表	8.0.8
Docker	19.03.12
Kubeflow	1.2
Kubernetes	1.18.9
NetApp Trident	21.01.2
NVIDIA DeepOps	提交时主分支的 Trident 部署功能 "61898cdfda"；从 21.03 版开始提供所有其他功能

支持

NetApp 不为 Apache Airflow ， Docker ， Kubeflow ， Kubernetes 或 NVIDIA DeepOps 提供企业级支持。如果您对具有与 NetApp AI 控制平台解决方案类似的功能的完全受支持的解决方案感兴趣， "请联系 NetApp" 关于 NetApp 与合作伙伴共同提供的完全受支持的 AI/ML 解决方案。

Kubernetes 部署

本节介绍部署 Kubernetes 集群以实施 NetApp AI 控制平面解决方案时必须完成的任务。如果您已有 Kubernetes 集群，则只要您运行的是 Kubernetes 和 NetApp Trident 支持的 Kubernetes 版本，就可以跳过本节。有关 Kubeflow 支持的 Kubernetes 版本列表，请参见 "Kubeflow 官方文档"。有关 Trident 支持的 Kubernetes 版本列表，请参见 "Trident 文档"。

对于采用采用 NVIDIA GPU 的裸机节点的内部 Kubernetes 部署， NetApp 建议使用 NVIDIA 的 DeepOps Kubernetes 部署工具。本节概述了使用 DeepOps 部署 Kubernetes 集群的过程。

前提条件

在执行本节所述的部署练习之前，我们假定您已执行以下任务：

1. 您已按照标准配置说明配置任何裸机 Kubernetes 节点（例如，属于 ONTAP AI POD 的 NVIDIA DGX 系统）。
2. 您已在所有 Kubernetes 主节点和工作节点以及部署跳转主机上安装受支持的操作系统。有关 DeepOps 支持的操作系统列表，请参见 "DeepOps GitHub 站点"。

使用 NVIDIA DeepOps 安装和配置 Kubernetes

要使用 NVIDIA DeepOps 部署和配置 Kubernetes 集群，请从部署跳转主机执行以下任务：

1. 按照上的说明下载 NVIDIA DeepOps "Getting Started 页面" 在 NVIDIA DeepOps GitHub 站点上。

2. 按照上的说明在集群中部署 Kubernetes "Kubernetes 部署指南页面" 在 NVIDIA DeepOps GitHub 站点上。

NetApp Trident 部署和配置

NetApp Trident 部署和配置

本节介绍在 Kubernetes 集群中安装和配置 NetApp Trident 时必须完成的任务。

前提条件

在执行本节所述的部署练习之前，我们假定您已执行以下任务：

1. 您已有一个有效的 Kubernetes 集群，并且正在运行 Trident 支持的 Kubernetes 版本。有关支持的版本列表，请参见 "Trident 文档"。
2. 您已有一个可正常工作的 NetApp 存储设备，软件定义的实例或云存储服务， Trident 支持此服务。

安装 **Trident**

要在 Kubernetes 集群中安装和配置 NetApp Trident ，请从部署跳转主机执行以下任务：

1. 使用以下方法之一部署 Trident ：
   - 如果您使用 NVIDIA DeepOps 部署 Kubernetes 集群，则也可以使用 NVIDIA DeepOps 在 Kubernetes 集群中部署 Trident 。要使用 DeepOps 部署 Trident ，请按照 "Trident 部署说明" 在 NVIDIA DeepOps GitHub 站点上。
   - 如果您未使用 NVIDIA DeepOps 部署 Kubernetes 集群，或者您只是希望手动部署 Trident ，则可以按照部署 Trident "部署说明" 在 Trident 文档中。有关如何配置的详细信息、请务必至少创建一个Trident后端和一个Kubernetes StorageClass "后端" 和 "StorageClasses" 请参见NetApp文档中链接的小节。

> 如果要在 ONTAP AI POD 上部署 NetApp AI 控制平台解决方案，请参见 "ONTAP AI 部署的 Trident 后端示例" 有关您可能希望创建和的不同 Trident 后端的一些示例，请参见 "适用于 ONTAP AI 部署的 Kubernetes Storageclasses 示例" 有关可能要创建的不同 Kubernetes StorageClasses 的一些示例，请参见。

ONTAP AI 部署的 Trident 后端示例

在使用 Trident 在 Kubernetes 集群中动态配置存储资源之前，必须先创建一个或多个 Trident 后端。以下示例展示了在 ONTAP AI POD 上部署 NetApp AI 控制平台解决方案时可能需要创建的不同类型的后端。有关后端的详细信息，请参见 "Trident 文档"。

1. NetApp 建议为要在 NetApp AFF 系统上使用的每个数据 LIF （提供数据访问的逻辑网络接口）创建一个启用了 FlexGroup 的 Trident 后端。这样，您可以在 LIF 之间平衡卷挂载

   以下示例命令显示了为与同一 ONTAP Storage Virtual Machine （ SVM ）关联的两个不同数据 LIF 创建两个启用了 FlexGroup 的 Trident 后端。这些后端使用 `ontap-nas-flexgroup` 存储驱动程序。ONTAP 支持两种主要数据卷类型： FlexVol 和 FlexGroup 。FlexVol 卷具有大小限制（截至本文撰写时，最大大小取决于特定部署）。另一方面， FlexGroup 卷可以线性扩展到高达 20 PB 和 4000 亿个文件，从而提供一个可显著简化数据管理的命名空间。因此， FlexGroup 卷最适合依赖大量数据的 AI 和 ML 工作负载。

如果您使用的是少量数据，并且希望使用 FlexVol 卷而不是 FlexGroup 卷，则可以创建使用 `ontap-NAS` 存储驱动程序而不是 `ontap-nas-flexgroup` 存储驱动程序的 Trident 后端。

```
$ cat << EOF > ./trident-backend-ontap-ai-flexgroups-iface1.json
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "backendName": "ontap-ai-flexgroups-iface1",
    "managementLIF": "10.61.218.100",
    "dataLIF": "192.168.11.11",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexgroups-
iface1.json -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
UUID                 | STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
$ cat << EOF > ./trident-backend-ontap-ai-flexgroups-iface2.json
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "backendName": "ontap-ai-flexgroups-iface2",
    "managementLIF": "10.61.218.100",
    "dataLIF": "192.168.12.12",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexgroups-
iface2.json -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
UUID                 | STATE  | VOLUMES |
+----------------------------+---------------------
```

```
+-------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+-------------------------------------+--------+---------+
$ tridentctl get backend -n trident
+----------------------------+---------------------
+-------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
UUID                 | STATE  | VOLUMES |
+----------------------------+---------------------
+-------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+-------------------------------------+--------+---------+
```

2. NetApp 还建议创建一个或多个启用了 FlexVol 的 Trident 后端。如果您使用 FlexGroup 卷来训练数据集存储，则可能需要使用 FlexVol 卷来存储结果，输出，调试信息等。如果要使用 FlexVol 卷，必须创建一个或多个启用了 FlexVol 的 Trident 后端。下面的示例命令显示了如何创建一个使用单个数据 LIF 且已启用 FlexVol 的 Trident 后端。

```
$ cat << EOF > ./trident-backend-ontap-ai-flexvols.json
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap-ai-flexvols",
    "managementLIF": "10.61.218.100",
    "dataLIF": "192.168.11.11",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexvols.json -n
trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |                 UUID
| STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexvols          | ontap-nas           | 52bdb3b1-13a5-4513-
a9c1-52a69657fabe | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
$ tridentctl get backend -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |                 UUID
| STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexvols          | ontap-nas           | 52bdb3b1-13a5-4513-
a9c1-52a69657fabe | online |       0 |
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
```

适用于 **ONTAP AI** 部署的 **Kubernetes StorageClasses** 示例

在使用 Trident 在 Kubernetes 集群中动态配置存储资源之前，必须创建一个或多个 Kubernetes StorageClasses 。以下示例展示了在 ONTAP AI POD 上部署 NetApp AI 控制

平面解决方案时可能需要创建的不同类型的 StorageClasses 。有关 StorageClasses 的详细信息，请参见 "Trident 文档"。

1. NetApp 建议为在一节中创建的每个启用了 FlexGroup 的 Trident 后端创建一个单独的 StorageClass "ONTAP AI 部署的 Trident 后端示例"，步骤 1 。通过这些粒度级 StorageClasses ，您可以将与特定 LIF （创建 Trident 后端时指定的 LIF ）相对应的 NFS 挂载添加为 StorageClass 规范文件中指定的特定后端。下面的示例命令显示了两个 StorageClasses 的创建过程，这两个 StorageClasses 对应于在部分中创建的两个示例后端 "ONTAP AI 部署的 Trident 后端示例"，步骤 1 。有关 StorageClasses 的详细信息，请参见 "Trident 文档"。

   为了在删除相应的 PersistentVolumeClaim （ PVC ）时不删除永久性卷，以下示例使用了 `reClaimPolicy` 值 `Retain` 。有关 `re"claimPolicy` " 字段的详细信息，请参见相关官员 "Kubernetes 文档"。

```
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain-iface1.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain-iface1
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
  storagePools: "ontap-ai-flexgroups-iface1:.*"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain-
iface1.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain-iface1 created
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain-iface2.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain-iface2
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
  storagePools: "ontap-ai-flexgroups-iface2:.*"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain-
iface2.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain-iface2 created
$ kubectl get storageclass
NAME                                PROVISIONER         AGE
ontap-ai-flexgroups-retain-iface1   netapp.io/trident   0m
ontap-ai-flexgroups-retain-iface2   netapp.io/trident   0m
```

2. NetApp 还建议创建一个与您在部分中创建的启用了 FlexVol 的 Trident 后端对应的 StorageClass "ONTAP AI 部署的 Trident 后端示例"，步骤 2 。下面的示例命令显示了为 FlexVol 卷创建一个 StorageClass 的过程。

   在以下示例中，未在 StorageClass 定义文件中指定特定后端，因为仅创建了一个启用了 FlexVol 的 Trident 后端。使用 Kubernetes 管理使用此 StorageClass 的卷时， Trident 会尝试使用使用 `ontap-NAS` 驱动程序的任何可用后端。

```
$ cat << EOF > ./storage-class-ontap-ai-flexvols-retain.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexvols-retain
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexvols-retain.yaml
storageclass.storage.k8s.io/ontap-ai-flexvols-retain created
$ kubectl get storageclass
NAME                                PROVISIONER         AGE
ontap-ai-flexgroups-retain-iface1   netapp.io/trident   1m
ontap-ai-flexgroups-retain-iface2   netapp.io/trident   1m
ontap-ai-flexvols-retain            netapp.io/trident   0m
```

3. NetApp 还建议为 FlexGroup 卷创建通用存储类。以下示例命令显示了如何为 FlexGroup 卷创建一个通用 StorageClass 。

   请注意， StorageClass 定义文件中未指定特定后端。因此，在使用 Kubernetes 管理使用此 StorageClass 的卷时， Trident 会尝试使用使用 `ontap-nas-flexgroup` 驱动程序的任何可用后端。

```
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain created
$ kubectl get storageclass
NAME                                  PROVISIONER         AGE
ontap-ai-flexgroups-retain            netapp.io/trident   0m
ontap-ai-flexgroups-retain-iface1     netapp.io/trident   2m
ontap-ai-flexgroups-retain-iface2     netapp.io/trident   2m
ontap-ai-flexvols-retain              netapp.io/trident   1m
```

Kubeflow 部署

本节介绍在 Kubernetes 集群中部署 Kubeflow 必须完成的任务。

前提条件

在执行本节所述的部署练习之前，我们假定您已执行以下任务：

1. 您已有一个有效的 Kubernetes 集群，并且正在运行 Kubernetes 支持的版本。有关支持的版本列表，请参见 "Kubeflow 官方文档"。
2. 您已在 Kubernetes 集群中安装和配置 NetApp Trident ，如中所述 "Trident 部署和配置"。

设置默认 Kubernetes StorageClass

在部署 Kubeflow 之前，您必须在 Kubernetes 集群中指定一个默认 StorageClass 。Kubeflow 部署过程会尝试使用默认 StorageClass 配置新的永久性卷。如果未将任何 StorageClass 指定为默认 StorageClass ，则部署将失败。要在集群中指定默认 StorageClass ，请从部署跳转主机执行以下任务。如果已在集群中指定默认 StorageClass ，则可以跳过此步骤。

1. 将现有 StorageClasses 之一指定为默认 StorageClass 。以下示例命令显示了名为 `ontap-ai-FlexVols-Retain` 的 StorageClass 的默认 StorageClass 。

> `ontap-nas-flexgroup` Trident 后端类型的最小 PVC 大小相当大。默认情况下， KubeFlow 会尝试配置大小只有少数几 GB 的 PVC 。因此，在部署 Kubeflow 时，不应将利用 `ontap-nas-flexgroup` 后端类型的 StorageClass 指定为默认 StorageClass 。

```
$ kubectl get sc
NAME                                  PROVISIONER             AGE
ontap-ai-flexgroups-retain            csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface1     csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface2     csi.trident.netapp.io   25h
ontap-ai-flexvols-retain              csi.trident.netapp.io   3s
$ kubectl patch storageclass ontap-ai-flexvols-retain -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
storageclass.storage.k8s.io/ontap-ai-flexvols-retain patched
$ kubectl get sc
NAME                                   PROVISIONER             AGE
ontap-ai-flexgroups-retain             csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface1      csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface2      csi.trident.netapp.io   25h
ontap-ai-flexvols-retain (default)     csi.trident.netapp.io   54s
```

使用 NVIDIA DeepOps 部署 Kubeflow

NetApp 建议使用 NVIDIA DeepOps 提供的 Kubeflow 部署工具。要使用 DeepOps 部署工具在 Kubernetes 集群中部署 Kubeflow ，请从部署跳转主机执行以下任务。

> 或者，您也可以按照手动方式部署 Kubeflow "安装说明" 在 Kubeflow 官方文档中

1. 按照中的说明在集群中部署 Kubeflow "Kubeflow 部署说明" 在 NVIDIA DeepOps GitHub 站点上。
2. 记下 DeepOps Kubeflow 部署工具输出的 Kubeflow 信息板 URL 。

```
$ ./scripts/k8s/deploy_kubeflow.sh -x
…
INFO[0007] Applied the configuration Successfully!
filename="cmd/apply.go:72"
Kubeflow app installed to: /home/ai/kubeflow
It may take several minutes for all services to start. Run 'kubectl get
pods -n kubeflow' to verify
To remove (excluding CRDs, istio, auth, and cert-manager), run:
./scripts/k8s_deploy_kubeflow.sh -d
To perform a full uninstall : ./scripts/k8s_deploy_kubeflow.sh -D
Kubeflow Dashboard (HTTP NodePort): http://10.61.188.111:31380
```

3. 确认在 Kubeflow 命名空间中部署的所有 Pod 均显示 `Ststatus` of `running` ，并确认命名空间中部署的任何组件均未处于错误状态。可能需要几分钟时间，才能启动所有 Pod 。

```
$ kubectl get all -n kubeflow
NAME                                                          READY
```

```
STATUS    RESTARTS   AGE
pod/admission-webhook-bootstrap-stateful-set-0                  1/1
Running   0          95s
pod/admission-webhook-deployment-6b89c84c98-vrtbh               1/1
Running   0          91s
pod/application-controller-stateful-set-0                       1/1
Running   0          98s
pod/argo-ui-5dcf5d8b4f-m2wn4                                    1/1
Running   0          97s
pod/centraldashboard-cf4874ddc-7hcr8                            1/1
Running   0          97s
pod/jupyter-web-app-deployment-685b455447-gjhh7                 1/1
Running   0          96s
pod/katib-controller-88c97d85c-kgq66                            1/1
Running   1          95s
pod/katib-db-8598468fd8-5jw2c                                   1/1
Running   0          95s
pod/katib-manager-574c8c67f9-wtrf5                              1/1
Running   1          95s
pod/katib-manager-rest-778857c989-fjbzn                         1/1
Running   0          95s
pod/katib-suggestion-bayesianoptimization-65df4d7455-qthmw      1/1
Running   0          94s
pod/katib-suggestion-grid-56bf69f597-98vwn                      1/1
Running   0          94s
pod/katib-suggestion-hyperband-7777b76cb9-9v6dq                 1/1
Running   0          93s
pod/katib-suggestion-nasrl-77f6f9458c-2qzxq                     1/1
Running   0          93s
pod/katib-suggestion-random-77b88b5c79-l64j9                    1/1
Running   0          93s
pod/katib-ui-7587c5b967-nd629                                   1/1
Running   0          95s
pod/metacontroller-0                                            1/1
Running   0          96s
pod/metadata-db-5dd459cc-swzkm                                  1/1
Running   0          94s
pod/metadata-deployment-6cf77db994-69fk7                        1/1
Running   3          93s
pod/metadata-deployment-6cf77db994-mpbjt                        1/1
Running   3          93s
pod/metadata-deployment-6cf77db994-xg7tz                        1/1
Running   3          94s
pod/metadata-ui-78f5b59b56-qb6kr                                1/1
Running   0          94s
pod/minio-758b769d67-llvdr                                      1/1
```

```
Running   0          91s
pod/ml-pipeline-5875b9db95-g8t2k                                1/1
Running   0          91s
pod/ml-pipeline-persistenceagent-9b69ddd46-bt9r9                1/1
Running   0          90s
pod/ml-pipeline-scheduledworkflow-7b8d756c76-7x56s              1/1
Running   0          90s
pod/ml-pipeline-ui-79ffd9c76-fcwpd                              1/1
Running   0          90s
pod/ml-pipeline-viewer-controller-deployment-5fdc87f58-b2t9r   1/1
Running   0          90s
pod/mysql-657f87857d-l5k9z                                      1/1
Running   0          91s
pod/notebook-controller-deployment-56b4f59bbf-8bvnr             1/1
Running   0          92s
pod/profiles-deployment-6bc745947-mrdkh                         2/2
Running   0          90s
pod/pytorch-operator-77c97f4879-hmlrv                           1/1
Running   0          92s
pod/seldon-operator-controller-manager-0                        1/1
Running   1          91s
pod/spartakus-volunteer-5fdfddb779-l7qkm                        1/1
Running   0          92s
pod/tensorboard-6544748d94-nh8b2                                1/1
Running   0          92s
pod/tf-job-dashboard-56f79c59dd-6w59t                           1/1
Running   0          92s
pod/tf-job-operator-79cbfd6dbc-rb58c                            1/1
Running   0          91s
pod/workflow-controller-db644d554-cwrnb                         1/1
Running   0          97s
NAME                                                   TYPE
CLUSTER-IP      EXTERNAL-IP   PORT(S)              AGE
service/admission-webhook-service                      ClusterIP
10.233.51.169   <none>        443/TCP              97s
service/application-controller-service                 ClusterIP
10.233.4.54     <none>        443/TCP              98s
service/argo-ui                                        NodePort
10.233.47.191   <none>        80:31799/TCP         97s
service/centraldashboard                               ClusterIP
10.233.8.36     <none>        80/TCP               97s
service/jupyter-web-app-service                        ClusterIP
10.233.1.42     <none>        80/TCP               97s
service/katib-controller                               ClusterIP
10.233.25.226   <none>        443/TCP              96s
service/katib-db                                       ClusterIP
```

```
10.233.33.151   <none>        3306/TCP            97s
service/katib-manager                                ClusterIP
10.233.46.239   <none>        6789/TCP            96s
service/katib-manager-rest                           ClusterIP
10.233.55.32    <none>        80/TCP              96s
service/katib-suggestion-bayesianoptimization        ClusterIP
10.233.49.191   <none>        6789/TCP            95s
service/katib-suggestion-grid                        ClusterIP
10.233.9.105    <none>        6789/TCP            95s
service/katib-suggestion-hyperband                   ClusterIP
10.233.22.2     <none>        6789/TCP            95s
service/katib-suggestion-nasrl                       ClusterIP
10.233.63.73    <none>        6789/TCP            95s
service/katib-suggestion-random                      ClusterIP
10.233.57.210   <none>        6789/TCP            95s
service/katib-ui                                     ClusterIP
10.233.6.116    <none>        80/TCP              96s
service/metadata-db                                  ClusterIP
10.233.31.2     <none>        3306/TCP            96s
service/metadata-service                             ClusterIP
10.233.27.104   <none>        8080/TCP            96s
service/metadata-ui                                  ClusterIP
10.233.57.177   <none>        80/TCP              96s
service/minio-service                                ClusterIP
10.233.44.90    <none>        9000/TCP            94s
service/ml-pipeline                                  ClusterIP
10.233.41.201   <none>        8888/TCP,8887/TCP   94s
service/ml-pipeline-tensorboard-ui                   ClusterIP
10.233.36.207   <none>        80/TCP              93s
service/ml-pipeline-ui                               ClusterIP
10.233.61.150   <none>        80/TCP              93s
service/mysql                                        ClusterIP
10.233.55.117   <none>        3306/TCP            94s
service/notebook-controller-service                  ClusterIP
10.233.10.166   <none>        443/TCP             95s
service/profiles-kfam                                ClusterIP
10.233.33.79    <none>        8081/TCP            92s
service/pytorch-operator                             ClusterIP
10.233.37.112   <none>        8443/TCP            95s
service/seldon-operator-controller-manager-service   ClusterIP
10.233.30.178   <none>        443/TCP             92s
service/tensorboard                                  ClusterIP
10.233.58.151   <none>        9000/TCP            94s
service/tf-job-dashboard                             ClusterIP
10.233.4.17     <none>        80/TCP              94s
service/tf-job-operator                              ClusterIP
```

```
10.233.60.32    <none>        8443/TCP            94s
service/webhook-server-service                      ClusterIP
10.233.32.167   <none>        443/TCP             87s
NAME                                                          READY   UP-
TO-DATE   AVAILABLE   AGE
deployment.apps/admission-webhook-deployment                  1/1     1
1           97s
deployment.apps/argo-ui                                       1/1     1
1           97s
deployment.apps/centraldashboard                              1/1     1
1           97s
deployment.apps/jupyter-web-app-deployment                    1/1     1
1           97s
deployment.apps/katib-controller                              1/1     1
1           96s
deployment.apps/katib-db                                      1/1     1
1           97s
deployment.apps/katib-manager                                 1/1     1
1           96s
deployment.apps/katib-manager-rest                            1/1     1
1           96s
deployment.apps/katib-suggestion-bayesianoptimization         1/1     1
1           95s
deployment.apps/katib-suggestion-grid                         1/1     1
1           95s
deployment.apps/katib-suggestion-hyperband                    1/1     1
1           95s
deployment.apps/katib-suggestion-nasrl                        1/1     1
1           95s
deployment.apps/katib-suggestion-random                       1/1     1
1           95s
deployment.apps/katib-ui                                      1/1     1
1           96s
deployment.apps/metadata-db                                   1/1     1
1           96s
deployment.apps/metadata-deployment                           3/3     3
3           96s
deployment.apps/metadata-ui                                   1/1     1
1           96s
deployment.apps/minio                                         1/1     1
1           94s
deployment.apps/ml-pipeline                                   1/1     1
1           94s
deployment.apps/ml-pipeline-persistenceagent                  1/1     1
1           93s
deployment.apps/ml-pipeline-scheduledworkflow                 1/1     1
```

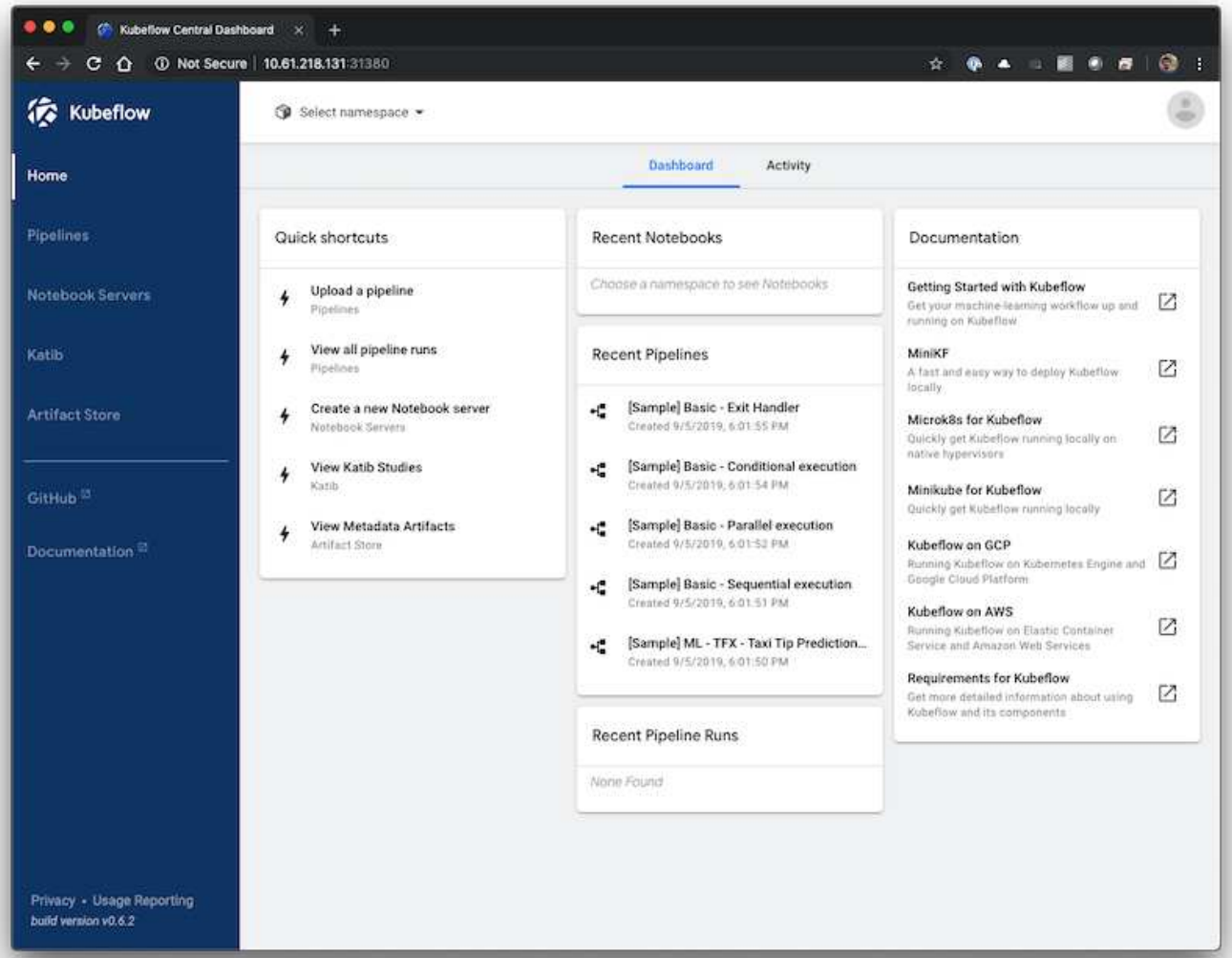
```
1           93s
deployment.apps/ml-pipeline-ui                                1/1     1
1           93s
deployment.apps/ml-pipeline-viewer-controller-deployment   1/1     1
1           93s
deployment.apps/mysql                                         1/1     1
1           94s
deployment.apps/notebook-controller-deployment                1/1     1
1           95s
deployment.apps/profiles-deployment                           1/1     1
1           92s
deployment.apps/pytorch-operator                              1/1     1
1           95s
deployment.apps/spartakus-volunteer                           1/1     1
1           94s
deployment.apps/tensorboard                                   1/1     1
1           94s
deployment.apps/tf-job-dashboard                              1/1     1
1           94s
deployment.apps/tf-job-operator                               1/1     1
1           94s
deployment.apps/workflow-controller                           1/1     1
1           97s
NAME
DESIRED   CURRENT   READY   AGE
replicaset.apps/admission-webhook-deployment-6b89c84c98              1
1         1       97s
replicaset.apps/argo-ui-5dcf5d8b4f                                   1
1         1       97s
replicaset.apps/centraldashboard-cf4874ddc                           1
1         1       97s
replicaset.apps/jupyter-web-app-deployment-685b455447                1
1         1       97s
replicaset.apps/katib-controller-88c97d85c                           1
1         1       96s
replicaset.apps/katib-db-8598468fd8                                  1
1         1       97s
replicaset.apps/katib-manager-574c8c67f9                             1
1         1       96s
replicaset.apps/katib-manager-rest-778857c989                        1
1         1       96s
replicaset.apps/katib-suggestion-bayesianoptimization-65df4d7455     1
1         1       95s
replicaset.apps/katib-suggestion-grid-56bf69f597                     1
1         1       95s
replicaset.apps/katib-suggestion-hyperband-7777b76cb9                1
```

```
1         1       95s
replicaset.apps/katib-suggestion-nasrl-77f6f9458c                    1
1         1       95s
replicaset.apps/katib-suggestion-random-77b88b5c79                   1
1         1       95s
replicaset.apps/katib-ui-7587c5b967                                  1
1         1       96s
replicaset.apps/metadata-db-5dd459cc                                 1
1         1       96s
replicaset.apps/metadata-deployment-6cf77db994                       3
3         3       96s
replicaset.apps/metadata-ui-78f5b59b56                               1
1         1       96s
replicaset.apps/minio-758b769d67                                     1
1         1       93s
replicaset.apps/ml-pipeline-5875b9db95                               1
1         1       93s
replicaset.apps/ml-pipeline-persistenceagent-9b69ddd46               1
1         1       92s
replicaset.apps/ml-pipeline-scheduledworkflow-7b8d756c76             1
1         1       91s
replicaset.apps/ml-pipeline-ui-79ffd9c76                             1
1         1       91s
replicaset.apps/ml-pipeline-viewer-controller-deployment-5fdc87f58   1
1         1       91s
replicaset.apps/mysql-657f87857d                                     1
1         1       92s
replicaset.apps/notebook-controller-deployment-56b4f59bbf            1
1         1       94s
replicaset.apps/profiles-deployment-6bc745947                        1
1         1       91s
replicaset.apps/pytorch-operator-77c97f4879                          1
1         1       94s
replicaset.apps/spartakus-volunteer-5fdfddb779                       1
1         1       94s
replicaset.apps/tensorboard-6544748d94                               1
1         1       93s
replicaset.apps/tf-job-dashboard-56f79c59dd                          1
1         1       93s
replicaset.apps/tf-job-operator-79cbfd6dbc                           1
1         1       93s
replicaset.apps/workflow-controller-db644d554                        1
1         1       97s
NAME                                                        READY   AGE
statefulset.apps/admission-webhook-bootstrap-stateful-set   1/1     97s
statefulset.apps/application-controller-stateful-set        1/1     98s
```

```
statefulset.apps/metacontroller                                  1/1     98s
statefulset.apps/seldon-operator-controller-manager              1/1     92s
$ kubectl get pvc -n kubeflow
NAME                STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS                AGE
katib-mysql         Bound    pvc-b07f293e-d028-11e9-9b9d-00505681a82d
10Gi       RWO              ontap-ai-flexvols-retain   27m
metadata-mysql      Bound    pvc-b0f3f032-d028-11e9-9b9d-00505681a82d
10Gi       RWO              ontap-ai-flexvols-retain   27m
minio-pv-claim      Bound    pvc-b22727ee-d028-11e9-9b9d-00505681a82d
20Gi       RWO              ontap-ai-flexvols-retain   27m
mysql-pv-claim      Bound    pvc-b2429afd-d028-11e9-9b9d-00505681a82d
20Gi       RWO              ontap-ai-flexvols-retain   27m
```

4. 在 Web 浏览器中，通过导航到步骤 2 中记下的 URL 来访问 Kubeflow 中央信息板。

   默认用户名为 `admin@kubeflow.org` ，默认密码为 `1231234` 。要创建其他用户，请按照中的说明进行操作 "Kubeflow 官方文档"。



Kubeflow 操作和任务示例

本节包括您可能希望使用 Kubeflow 执行的各种操作和任务的示例。

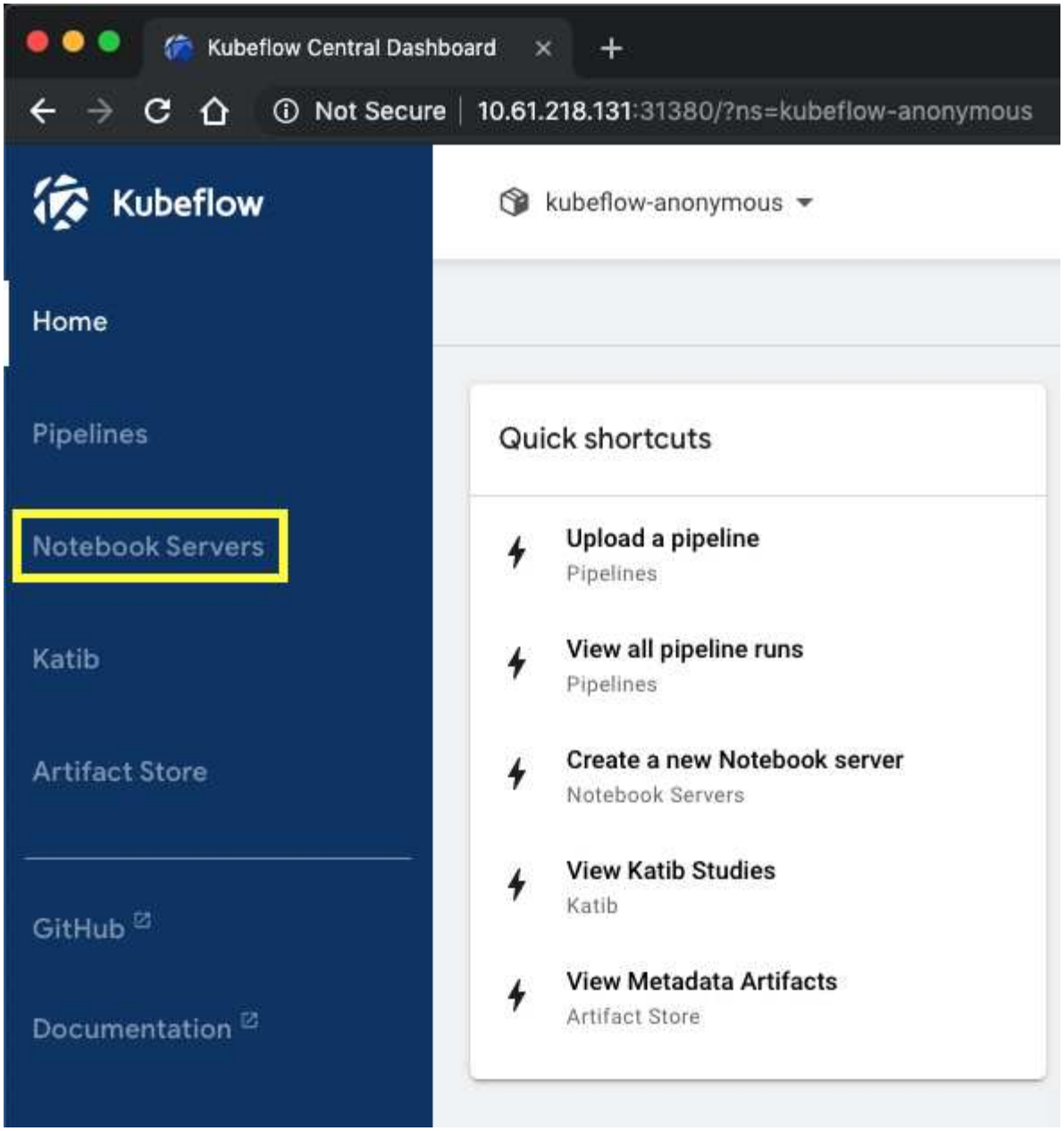
Kubeflow 操作和任务示例

本节包括您可能希望使用 Kubeflow 执行的各种操作和任务的示例。

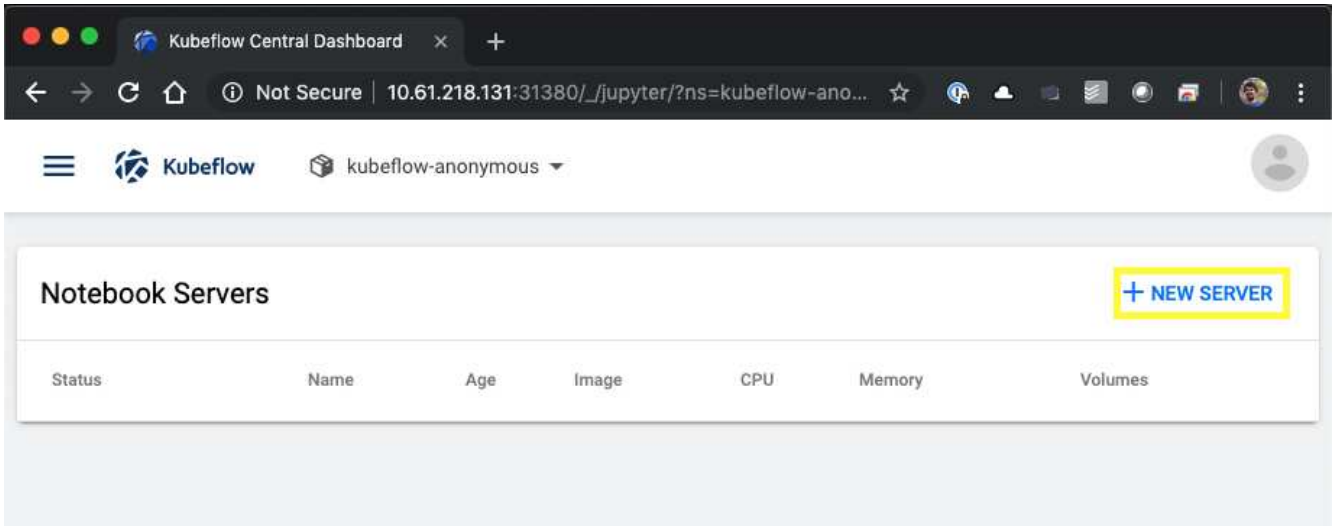
为数据科学家或开发人员配置 Jupyter 笔记本电脑工作空间

Kubeflow 能够快速配置新的 Jupyter 笔记本电脑服务器，以充当数据科学家工作空间。要使用 Kubeflow 配置新的 Jupyter 笔记本电脑服务器，请执行以下任务。有关 Kubeflow 上下文中 Jupyter 笔记本电脑的详细信息，请参见 "Kubeflow 官方文档"。

1. 从 Kubeflow 中央信息板中，单击主菜单中的 Notebook Servers 以导航到 Jupyter 笔记本电脑服务器管理页面。

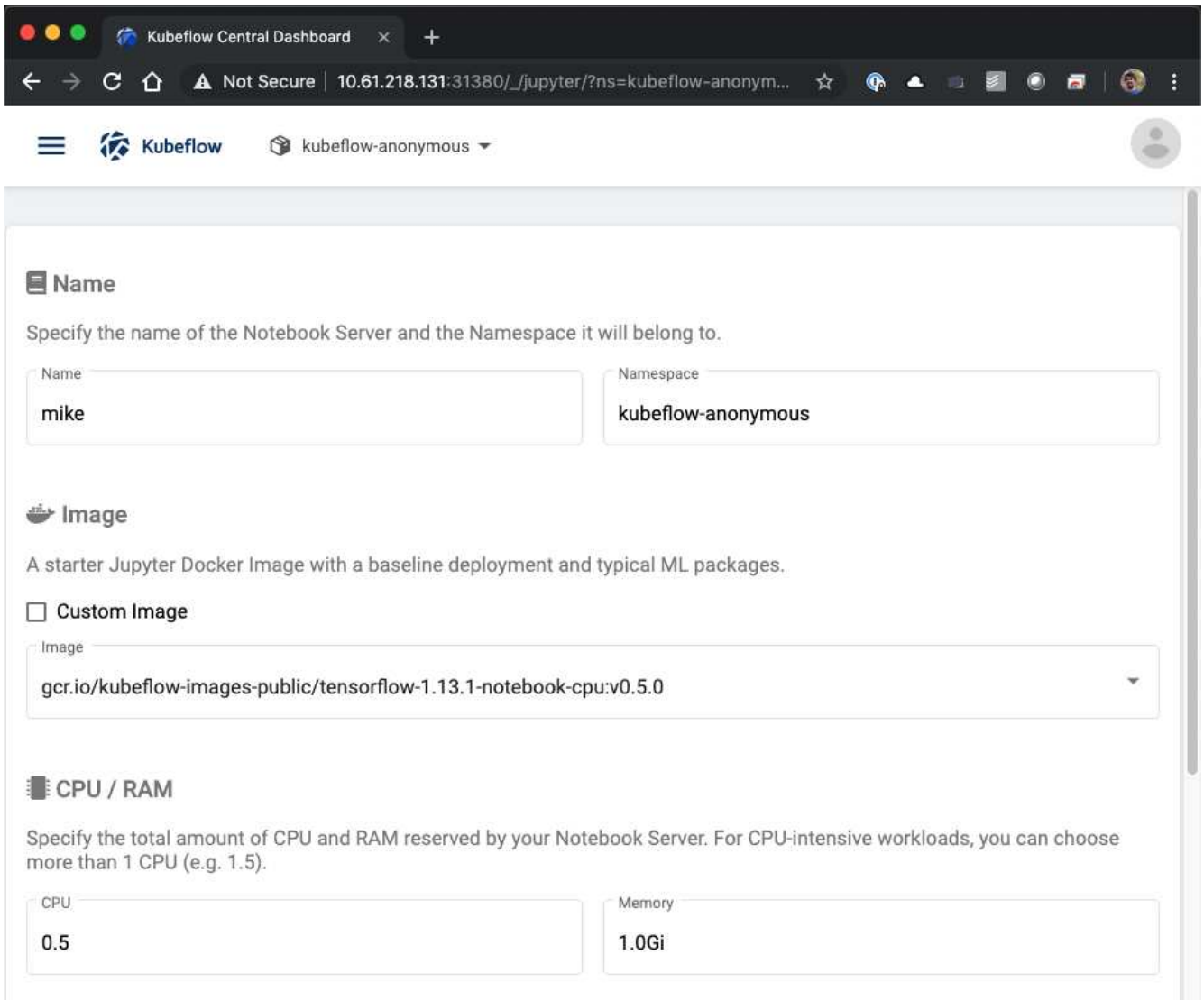


2. 单击新服务器以配置新的 Jupyter 笔记本电脑服务器。

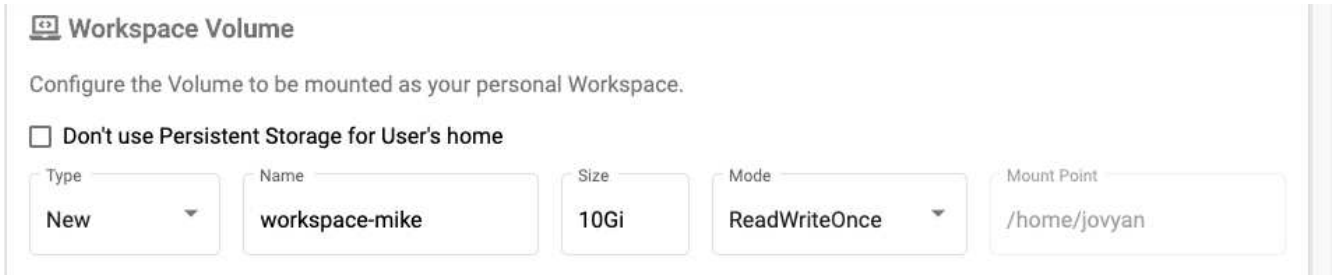


3. 为新服务器指定一个名称，选择希望服务器基于的 Docker 映像，并指定服务器要预留的 CPU 和 RAM 量。如果命名空间字段为空，请使用页面标题中的选择命名空间菜单选择命名空间。然后， Namespace 字段将自动填充所选命名空间。

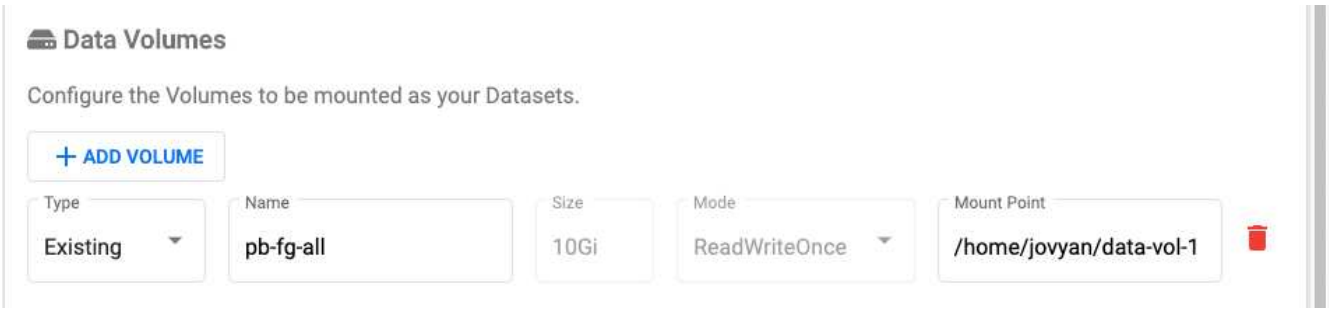
   在以下示例中，选择了 `kubeflow-anonymous` 命名空间。此外，还接受 Docker 映像， CPU 和 RAM 的默认值。



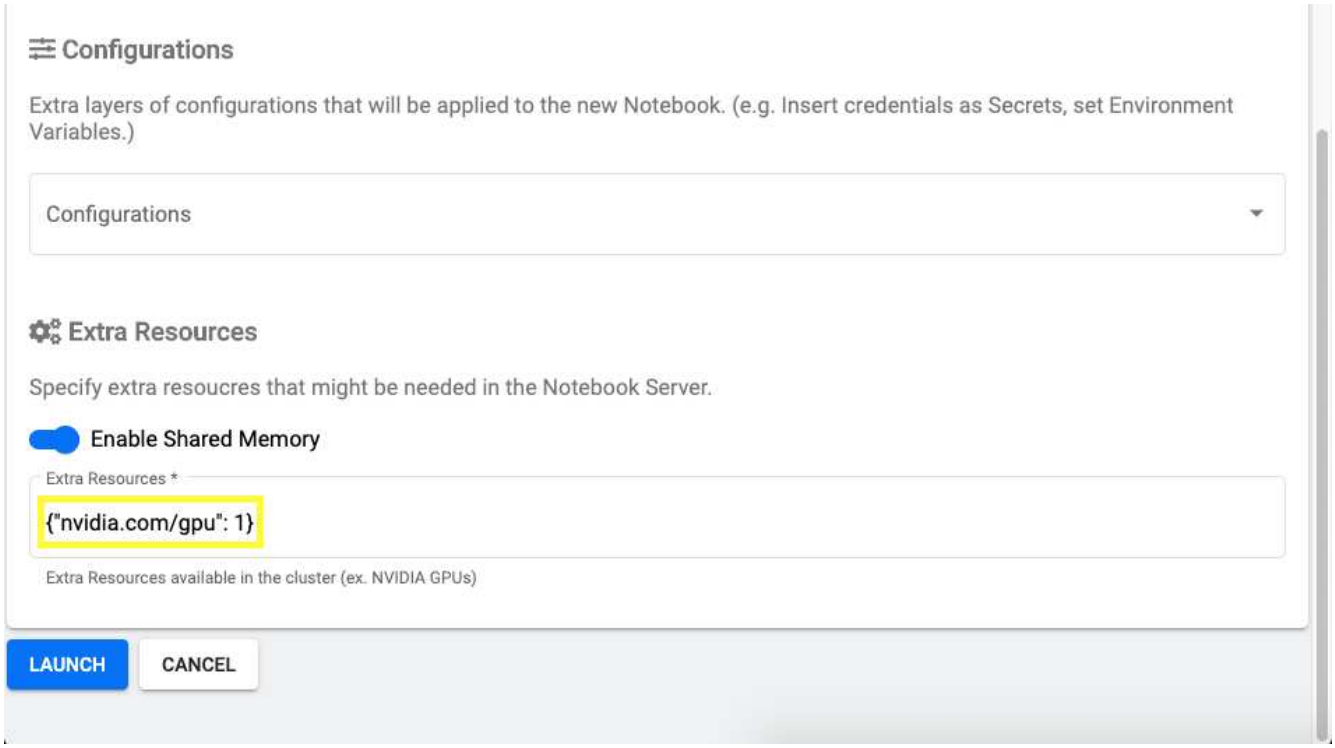
4. 指定工作空间卷详细信息。如果选择创建新卷，则会使用默认 StorageClass 配置该卷或 PVC 。因为在部分中，使用 Trident 的 StorageClass 被指定为默认 StorageClass "Kubeflow 部署"，卷或 PVC 配置有 Trident 。此卷会自动挂载为 Jupyter 笔记本电脑服务器容器中的默认工作空间。用户在服务器上创建但未保存到单独数据卷的任何笔记本电脑将自动保存到此工作空间卷。因此，这些笔记本电脑在重新启动后会持久存在。



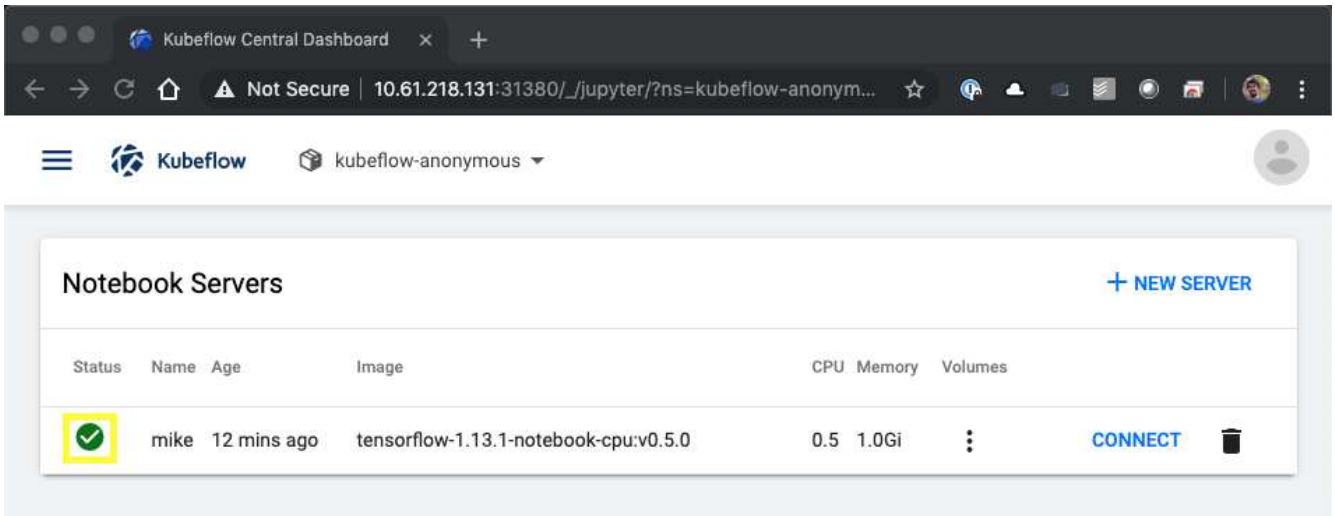
5. 添加数据卷。以下示例指定了一个名为 "pt-fg-all" 的现有 PVC 并接受默认挂载点。



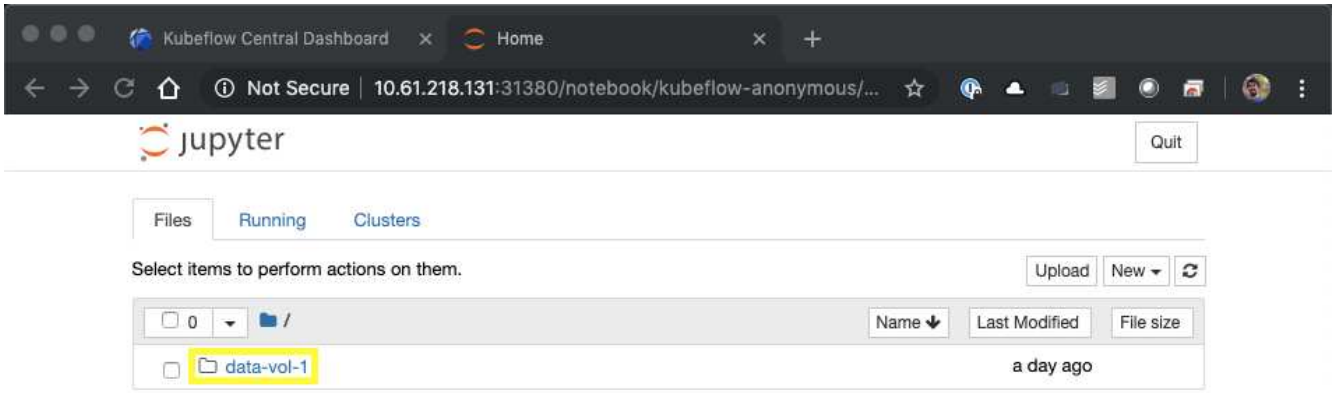
6. * 可选： * 请求将所需数量的 GPU 分配给您的笔记本服务器。在以下示例中，请求一个 GPU 。

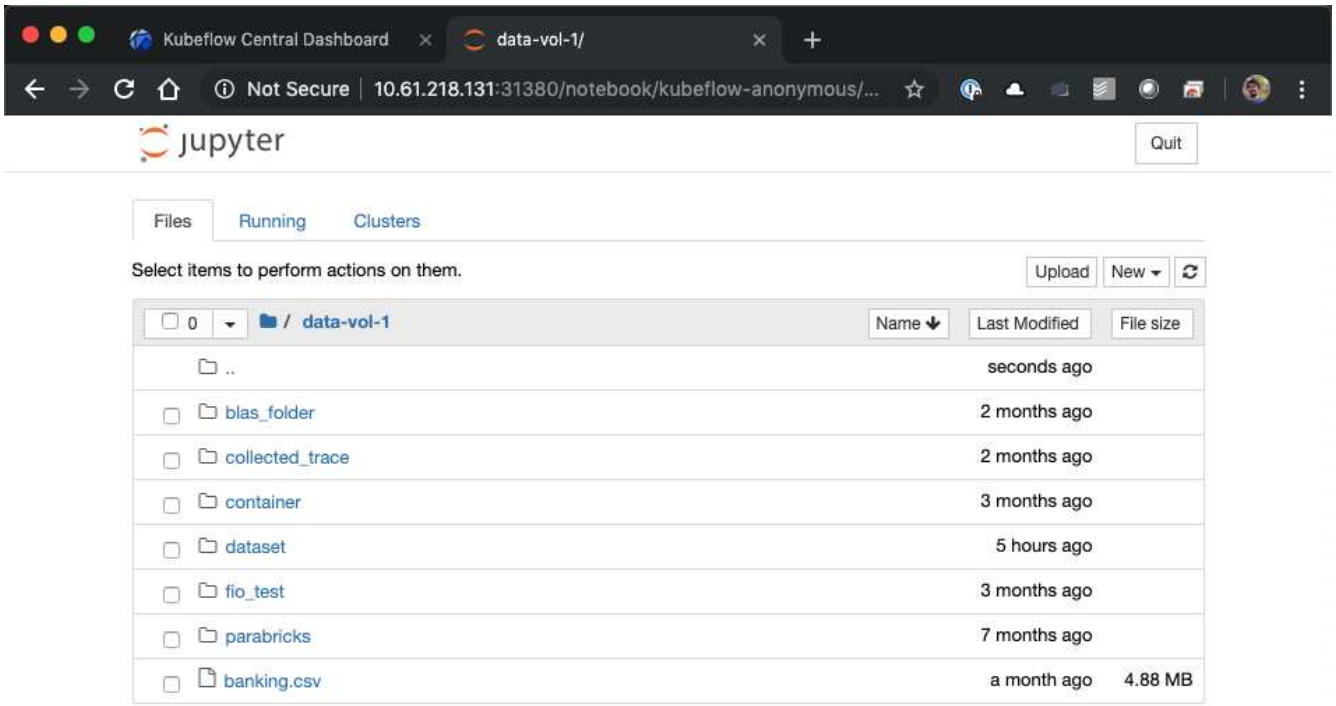


7. 单击启动以配置新的笔记本电脑服务器。
8. 等待笔记本电脑服务器完全配置完毕。如果您从未使用您指定的 Docker 映像配置服务器，则可能需要几分钟的时间，因为需要下载此映像。服务器配置完成后， Jupyter 笔记本电脑服务器管理页面上的状态列会显示一个绿色复选标记。



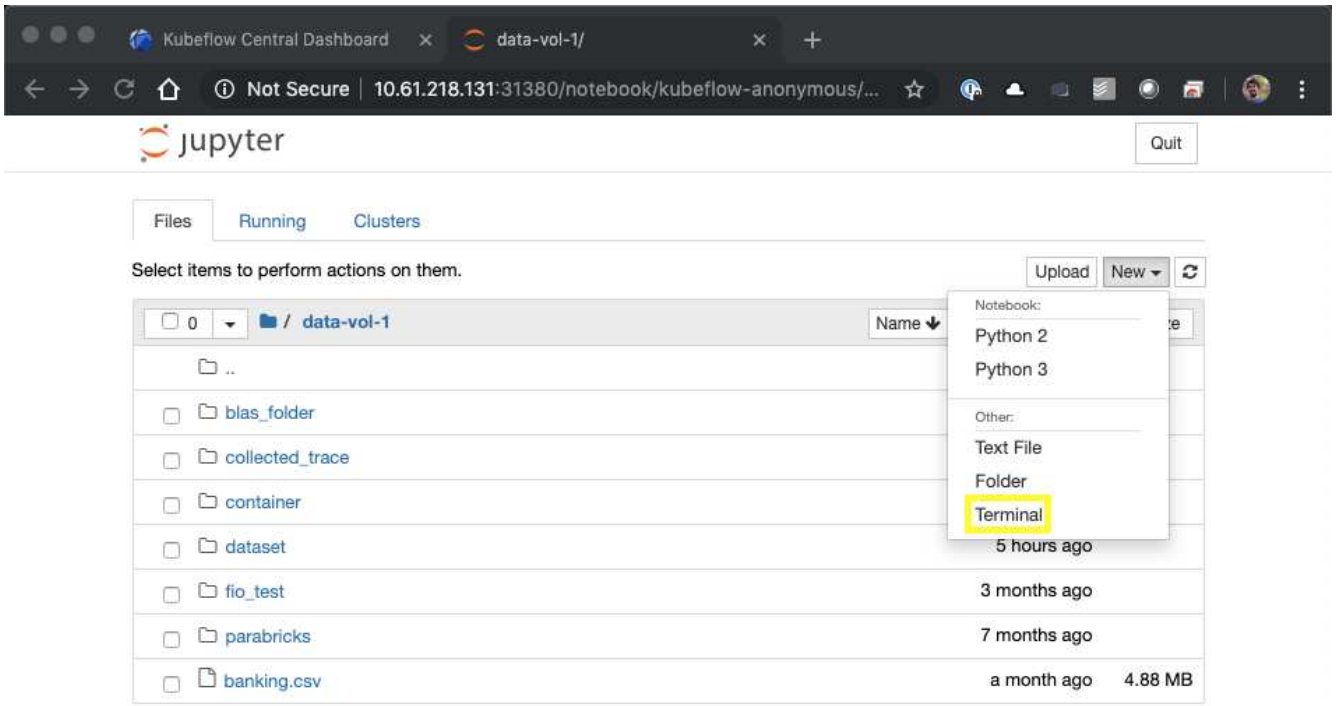
9. 单击连接以连接到新的服务器 Web 界面。
10. 确认步骤 6 中指定的数据集卷已挂载到服务器上。请注意，默认情况下，此卷会挂载在默认工作空间中。从用户的角度来看，这只是工作空间中的另一个文件夹。用户可能是数据科学家，而不是基础架构专家，因此使用此卷无需具备任何存储专业知识。

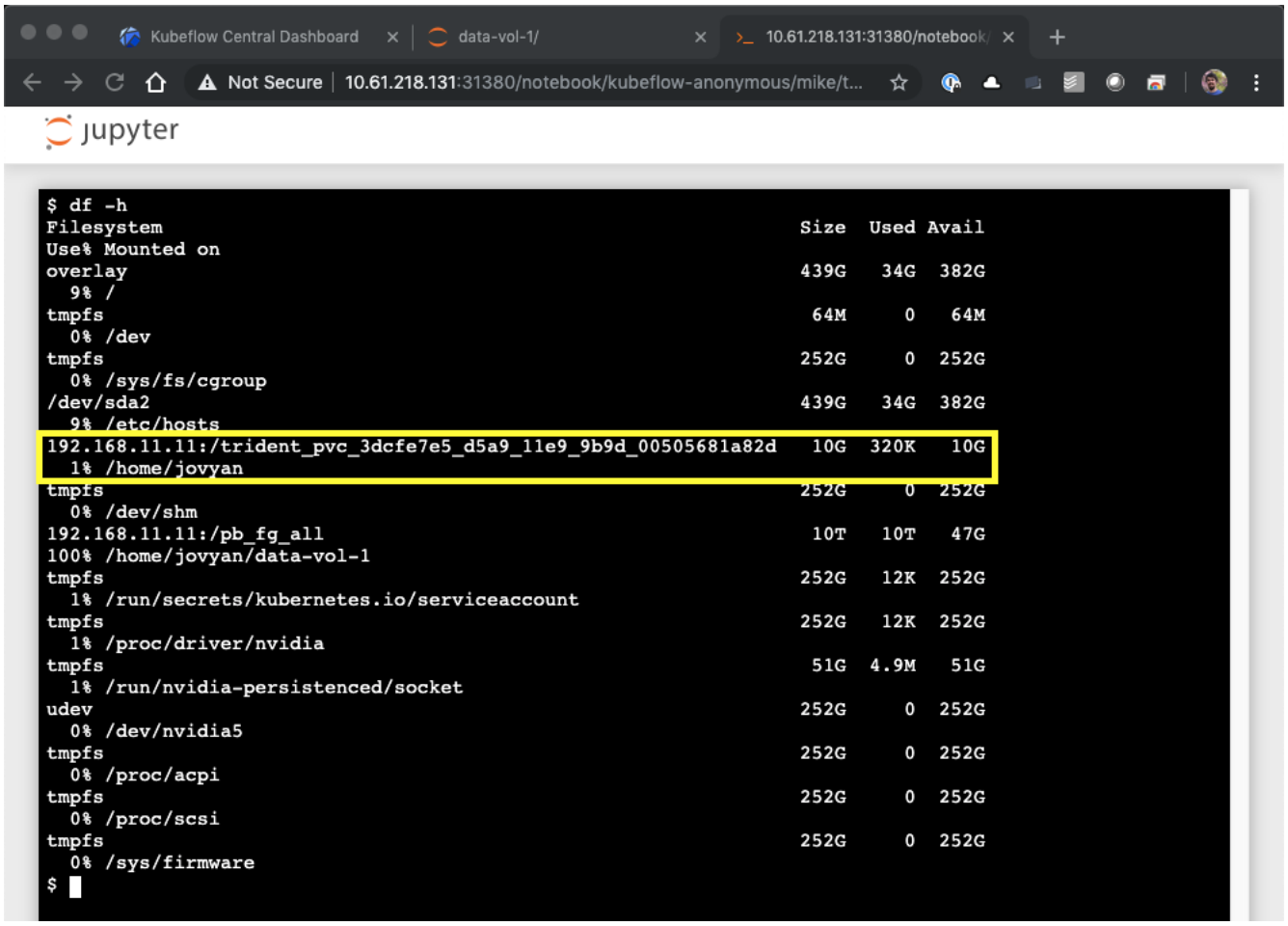




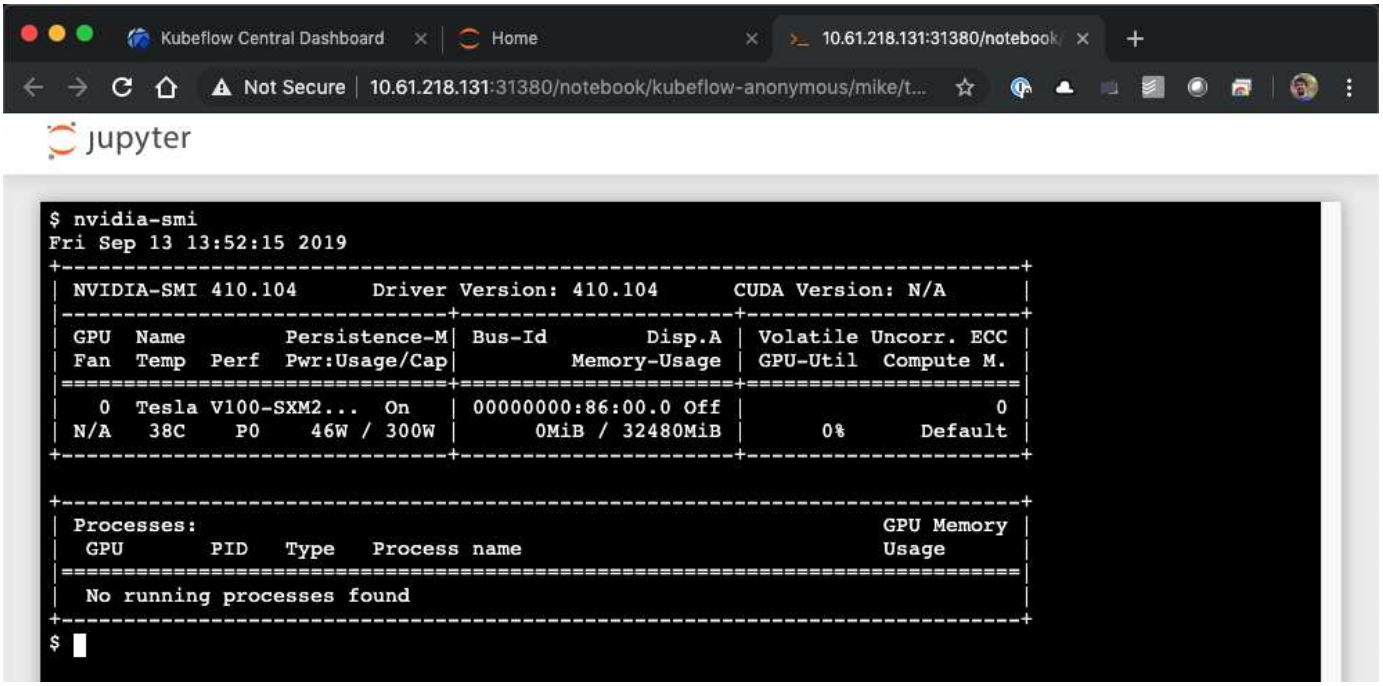
11. 打开一个终端，假设步骤 5 中请求了一个新卷，请执行 `df -h` 以确认已挂载新的 Trident 配置的永久性卷作为默认工作空间。

    默认工作空间目录是首次访问服务器的 Web 界面时显示的基目录。因此，使用 Web 界面创建的任何项目都会存储在此 Trident 配置的永久性卷上。





12. 使用终端运行 `nvidia-smi` 以确认为笔记本电脑服务器分配了正确数量的 GPU 。在以下示例中，已按照步骤 7 中的请求为笔记本电脑服务器分配一个 GPU 。



示例笔记本电脑和管道

。 "适用于 Kubernetes 的 NetApp 数据科学工具包" 可与 Kubeflow 结合使用。将 NetApp 数据科学工具包与 Kubeflow 结合使用具有以下优势：

- 数据科学家可以直接在 Jupyter 笔记本电脑中执行高级 NetApp 数据管理操作。
- 可以使用 Kubeflow 管道框架将高级 NetApp 数据管理操作整合到自动化工作流中。

请参见 "Kubeflow 示例" 有关将工具包与 Kubeflow 结合使用的详细信息，请参见 NetApp Data Science Toolkit GitHub 存储库中的一节。

Apache Airflow 部署

NetApp 建议在 Kubernetes 顶部运行 Apache Airflow 。本节介绍在 Kubernetes 集群中部署气流时必须完成的任务。

> 可以在 Kubernetes 以外的平台上部署 Airflow 。在 Kubernetes 以外的平台上部署气流不在此解决方案的范围内。

前提条件

在执行本节所述的部署练习之前，我们假定您已执行以下任务：

1. 您已有一个工作正常的 Kubernetes 集群。
2. 您已按照《 NetApp Trident 部署和配置》一节中所述在 Kubernetes 集群中安装和配置 NetApp Trident 。

安装 Helm

Airflow 可使用 Kubernetes 常用的软件包管理器 Helm 进行部署。在部署气流之前，必须在部署跳转主机上安装 Helm 。要在部署跳转主机上安装 Helm ，请按照 "安装说明" 在官方 Helm 文档中。

设置默认 Kubernetes StorageClass

在部署 Airflow 之前，您必须在 Kubernetes 集群中指定一个默认 StorageClass 。气流部署过程会尝试使用默认 StorageClass 配置新的永久性卷。如果未将任何 StorageClass 指定为默认 StorageClass ，则部署将失败。要在集群中指定默认 StorageClass ，请按照一节中所述的说明进行操作 "Kubeflow 部署"。如果已在集群中指定默认 StorageClass ，则可以跳过此步骤。

使用 Helm 部署气流

要使用 Helm 在 Kubernetes 集群中部署气流，请从部署跳转主机执行以下任务：

1. 按照说明使用 Helm 部署气流 "部署说明" 用于 Artifact Hub 上的官方气流图表。下面的示例命令显示了如何使用 Helm 部署气流。根据您的环境和所需配置，根据需要修改，添加和 / 或删除 `custom-values.yaml` 文件中的值。

```
$ cat << EOF > custom-values.yaml
```

```
#####################################
# Airflow - Common Configs
#####################################
airflow:
  ## the airflow executor type to use
  ##
  executor: "CeleryExecutor"
  ## environment variables for the web/scheduler/worker Pods (for
airflow configs)
  ##
  #
#####################################
# Airflow - WebUI Configs
#####################################
web:
  ## configs for the Service of the web Pods
  ##
  service:
    type: NodePort
#####################################
# Airflow - Logs Configs
#####################################
logs:
  persistence:
    enabled: true
#####################################
# Airflow - DAGs Configs
#####################################
dags:
  ## configs for the DAG git repository & sync container
  ##
  gitSync:
    enabled: true
    ## url of the git repository
    ##
    repo: "git@github.com:mboglesby/airflow-dev.git"
    ## the branch/tag/sha1 which we clone
    ##
    branch: master
    revision: HEAD
    ## the name of a pre-created secret containing files for ~/.ssh/
    ##
    ## NOTE:
    ## - this is ONLY RELEVANT for SSH git repos
    ## - the secret commonly includes files: id_rsa, id_rsa.pub,
known_hosts
```

```
    ## - known_hosts is NOT NEEDED if `git.sshKeyscan` is true
    ##
    sshSecret: "airflow-ssh-git-secret"
    ## the name of the private key file in your `git.secret`
    ##
    ## NOTE:
    ## - this is ONLY RELEVANT for PRIVATE SSH git repos
    ##
    sshSecretKey: id_rsa
    ## the git sync interval in seconds
    ##
    syncWait: 60
EOF
$ helm install airflow airflow-stable/airflow -n airflow --version 8.0.8
--values ./custom-values.yaml
...
Congratulations. You have just deployed Apache Airflow!
1. Get the Airflow Service URL by running these commands:
   export NODE_PORT=$(kubectl get --namespace airflow -o
jsonpath="{.spec.ports[0].nodePort}" services airflow-web)
   export NODE_IP=$(kubectl get nodes --namespace airflow -o
jsonpath="{.items[0].status.addresses[0].address}")
   echo http://$NODE_IP:$NODE_PORT/
2. Open Airflow in your web browser
```

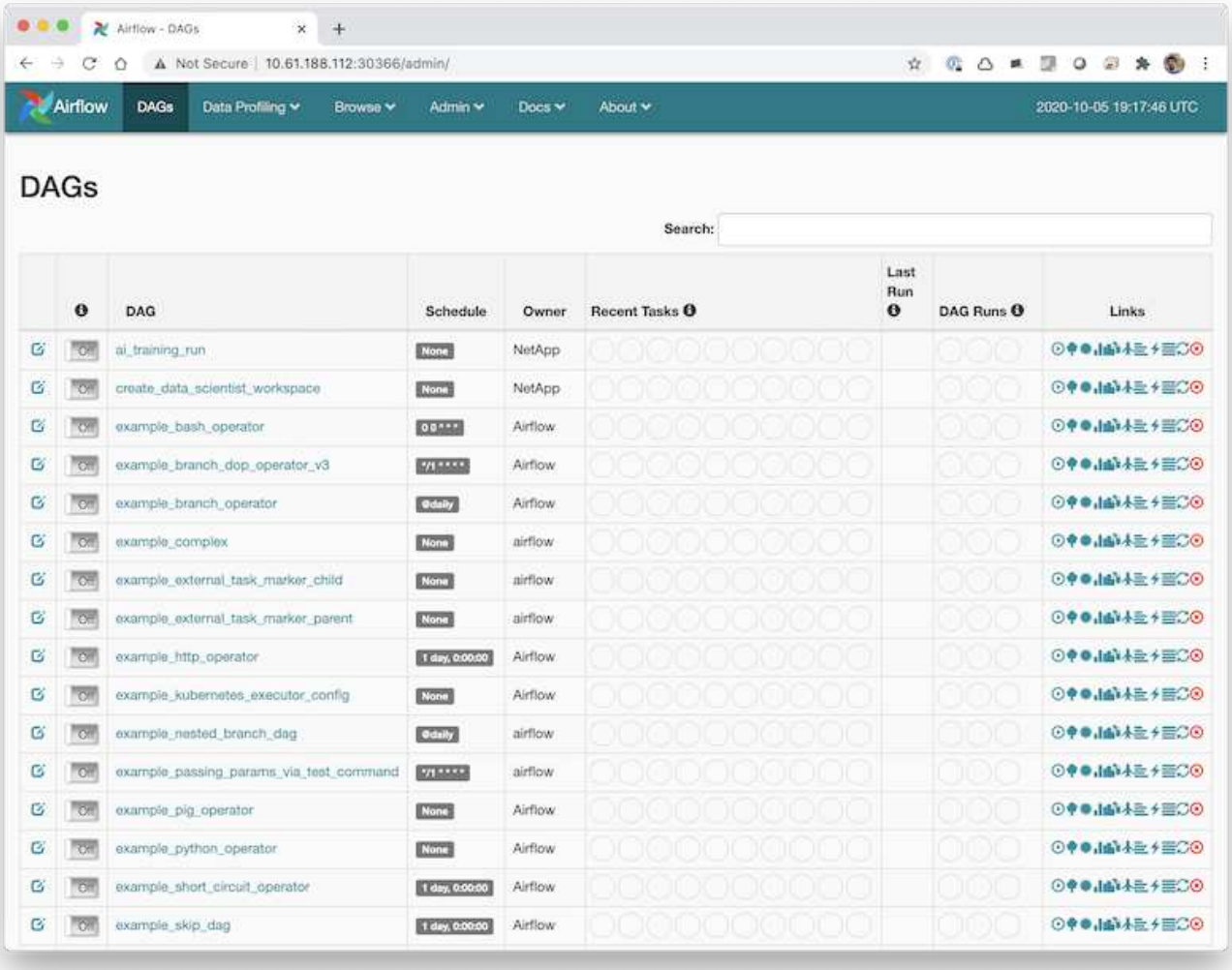
2. 确认所有气流 Pod 均已启动且正在运行。所有 POD 可能需要几分钟的时间才能启动。

```
$ kubectl -n airflow get pod
NAME                                READY   STATUS    RESTARTS   AGE
airflow-flower-b5656d44f-h8qjk      1/1     Running   0          2h
airflow-postgresql-0                1/1     Running   0          2h
airflow-redis-master-0              1/1     Running   0          2h
airflow-scheduler-9d95fcdf9-clf4b   2/2     Running   2          2h
airflow-web-59c94db9c5-z7rg4        1/1     Running   0          2h
airflow-worker-0                    2/2     Running   2          2h
```

3. 按照步骤 1 中使用 Helm 部署 Airflow 时控制台上印有的说明获取 Airflow Web 服务 URL 。

```
$ export NODE_PORT=$(kubectl get --namespace airflow -o
jsonpath="{.spec.ports[0].nodePort}" services airflow-web)
$ export NODE_IP=$(kubectl get nodes --namespace airflow -o
jsonpath="{.items[0].status.addresses[0].address}")
$ echo http://$NODE_IP:$NODE_PORT/
```

4. 确认您可以访问 Airflow Web 服务。



Apache 气流工作流示例

。 "适用于 Kubernetes 的 NetApp 数据科学工具包" 可与气流结合使用。通过将 NetApp 数据科学工具包与 Airflow 结合使用，您可以将 NetApp 数据管理操作整合到由 Airflow 协调的自动化工作流中。

请参见 "气流示例" 有关在 Airflow 中使用工具包的详细信息，请参见 NetApp Data Science Toolkit GitHub 存储库中的一节。

Trident 操作示例

本节包括您可能希望使用 Trident 执行的各种操作的示例。

导入现有卷

如果您的 NetApp 存储系统 / 平台上有要挂载到 Kubernetes 集群中的容器上但未与集群中的 PVC 绑定的现有卷

，则必须导入这些卷。您可以使用 Trident 卷导入功能导入这些卷。

以下示例命令显示了为在本节示例中创建的每个 Trident 后端导入相同卷 `PB_FG_ALL` 两次 "ONTAP AI 部署的 Trident 后端示例"，步骤 1 。通过以这种方式导入同一卷两次，您可以在不同 LIF 之间多次挂载此卷（现有 FlexGroup 卷），如一节所述 "ONTAP AI 部署的 Trident 后端示例"，步骤 1 。有关 PVCs 的详细信息，请参见 "Kubernetes 官方文档"。有关卷导入功能的详细信息，请参见 "Trident 文档"。

在示例 PVC 规范文件中指定了 `accessModes` 值 `ReadOnlyMany` 。有关 `accessMode` 字段的详细信息，请参见 "Kubernetes 官方文档"。

> 以下示例导入命令中指定的后端名称与在本节的示例中创建的后端相对应 "ONTAP AI 部署的 Trident 后端示例"，步骤 1 。以下示例 PVC 定义文件中指定的 StorageClass 名称与在本节的示例中创建的 StorageClasses 相对应 "适用于 ONTAP AI 部署的 Kubernetes StorageClasses 示例"，步骤 1 。

```
$ cat << EOF > ./pvc-import-pb_fg_all-iface1.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pb-fg-all-iface1
  namespace: default
spec:
  accessModes:
    - ReadOnlyMany
  storageClassName: ontap-ai-flexgroups-retain-iface1
EOF
$ tridentctl import volume ontap-ai-flexgroups-iface1 pb_fg_all -f ./pvc-
import-pb_fg_all-iface1.yaml -n trident
+------------------------------+--------
+----------------------------------+----------
+-----------------------------------------+--------+---------+
|          NAME                  |  SIZE  |       STORAGE CLASS
| PROTOCOL |             BACKEND UUID                         | STATE  |
MANAGED |
+------------------------------+--------
+----------------------------------+----------
+-----------------------------------------+--------+---------+
| default-pb-fg-all-iface1-7d9f1 | 10 TiB | ontap-ai-flexgroups-retain-
iface1 | file     | b74cbddb-e0b8-40b7-b263-b6da6dec0bdd | online | true
|
+------------------------------+--------
+----------------------------------+----------
+-----------------------------------------+--------+---------+
$ cat << EOF > ./pvc-import-pb_fg_all-iface2.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
```

```
  name: pb-fg-all-iface2
  namespace: default
spec:
  accessModes:
    - ReadOnlyMany
  storageClassName: ontap-ai-flexgroups-retain-iface2
EOF
$ tridentctl import volume ontap-ai-flexgroups-iface2 pb_fg_all -f ./pvc-
import-pb_fg_all-iface2.yaml -n trident
+--------------------------------+--------
+-----------------------------------+----------
+-------------------------------------------+--------+---------+
|          NAME                  |  SIZE  |       STORAGE CLASS
| PROTOCOL |             BACKEND UUID                        | STATE  |
MANAGED |
+--------------------------------+--------
+-----------------------------------+----------
+-------------------------------------------+--------+---------+
| default-pb-fg-all-iface2-85aee | 10 TiB | ontap-ai-flexgroups-retain-
iface2 | file     | 61814d48-c770-436b-9cb4-cf7ee661274d | online | true
|
+--------------------------------+--------
+-----------------------------------+----------
+-------------------------------------------+--------+---------+
$ tridentctl get volume -n trident
+------------------------------------+---------
+------------------------------------+----------
+--------------------------------------+--------+---------+
|               NAME               |  SIZE   |           STORAGE CLASS
| PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------+---------
+------------------------------------+----------
+--------------------------------------+--------+---------+
| default-pb-fg-all-iface1-7d9f1   | 10 TiB  | ontap-ai-flexgroups-retain-
iface1 | file     | b74cbddb-e0b8-40b7-b263-b6da6dec0bdd | online | true
|
| default-pb-fg-all-iface2-85aee   | 10 TiB  | ontap-ai-flexgroups-retain-
iface2 | file     | 61814d48-c770-436b-9cb4-cf7ee661274d | online | true
|
+------------------------------------+---------
+------------------------------------+----------
+--------------------------------------+--------+---------+
$ kubectl get pvc
NAME                 STATUS   VOLUME                              CAPACITY
ACCESS MODES   STORAGECLASS                        AGE
pb-fg-all-iface1     Bound    default-pb-fg-all-iface1-7d9f1
```

```
10995116277760   ROX            ontap-ai-flexgroups-retain-iface1   25h
pb-fg-all-iface2     Bound    default-pb-fg-all-iface2-85aee
10995116277760   ROX            ontap-ai-flexgroups-retain-iface2   25h
```

配置新卷

您可以使用 Trident 在 NetApp 存储系统或平台上配置新卷。以下示例命令显示了新 FlexVol 卷的配置。在此示例中，使用在一节的示例中创建的 StorageClass 配置卷 "适用于 ONTAP AI 部署的 Kubernetes StorageClasses 示例"，步骤 2 。

在以下示例 PVC 定义文件中指定了 `accessModes` 值 `ReadWriteMany` 。有关 `accessMode` 字段的详细信息，请参见 "Kubernetes 官方文档"。

```
$ cat << EOF > ./pvc-tensorflow-results.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: tensorflow-results
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  storageClassName: ontap-ai-flexvols-retain
EOF
$ kubectl create -f ./pvc-tensorflow-results.yaml
persistentvolumeclaim/tensorflow-results created
$ kubectl get pvc
NAME                              STATUS    VOLUME
CAPACITY         ACCESS MODES   STORAGECLASS                        AGE
pb-fg-all-iface1                  Bound     default-pb-fg-all-iface1-7d9f1
10995116277760   ROX            ontap-ai-flexgroups-retain-iface1   26h
pb-fg-all-iface2                  Bound     default-pb-fg-all-iface2-85aee
10995116277760   ROX            ontap-ai-flexgroups-retain-iface2   26h
tensorflow-results                Bound     default-tensorflow-results-
2fd60   1073741824       RWX            ontap-ai-flexvols-retain
25h
```

ONTAP AI 部署的高性能作业示例

本节包括在 ONTAP AI POD 上部署 Kubernetes 时可以执行的各种高性能作业的示例。

ONTAP AI 部署的高性能作业示例

本节包括在 ONTAP AI POD 上部署 Kubernetes 时可以执行的各种高性能作业的示例。

执行单节点 AI 工作负载

要在 Kubernetes 集群中执行单节点 AI 和 ML 作业，请从部署跳转主机执行以下任务。借助 Trident ，您可以快速轻松地使可能包含数 PB 数据的数据卷可供 Kubernetes 工作负载访问。要使此类数据卷可从 Kubernetes Pod 中访问，只需在 Pod 定义中指定 PVC 即可。此步骤是 Kubernetes 本机操作；不需要 NetApp 专业知识。

> 本节假定您已将尝试在 Kubernetes 集群中执行的特定 AI 和 ML 工作负载容器化（采用 Docker 容器格式）。

1. 以下示例命令显示了如何为使用 ImageNet 数据集的 TensorFlow 基准工作负载创建 Kubernetes 作业。有关 ImageNet 数据集的详细信息，请参见 "ImageNet 网站"。

   此示例作业请求八个 GPU ，因此可以在具有八个或更多 GPU 的单个 GPU 工作节点上运行。此示例作业可以在集群中提交，其中包含八个或更多 GPU 的工作节点不存在或当前占用另一个工作负载。如果是，则此作业将保持待定状态，直到此类辅助节点变为可用为止。

   此外，为了最大程度地提高存储带宽，包含所需训练数据的卷会在该作业创建的 POD 中挂载两次。此外，还会在 Pod 中挂载另一个卷。第二个卷将用于存储结果和指标。这些卷在作业定义中使用 PVC 的名称进行引用。有关 Kubernetes 作业的详细信息，请参见 "Kubernetes 官方文档"。

   在本示例作业创建的 Pod 中， `edium` 值为 `Memory` 的` emtyDir `m卷将挂载到 ` `/dev/shm` 。Docker 容器运行时自动创建的 ` /dev/shm` 虚拟卷的默认大小有时可能不足以满足 TensorFlow 的需求。按以下示例所示挂载 `emptyDir` 卷可提供足够大的 ` /dev/shm` 虚拟卷。有关 `emptyDir` 卷的详细信息，请参见 "Kubernetes 官方文档"。

   在此示例作业定义中指定的单个容器将获得 `securityContext > privileged` 值 `true` 。此值表示容器在主机上具有有效的 root 访问权限。在这种情况下使用此标注是因为要执行的特定工作负载需要 root 访问权限。具体而言，工作负载执行的清除缓存操作需要 root 访问权限。是否需要此 特权： `true` 标注取决于您要执行的特定工作负载的要求。

```
$ cat << EOF > ./netapp-tensorflow-single-imagenet.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-tensorflow-single-imagenet
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
```

```
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--dgx_version=dgx1", "--
num_devices=8"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-single-imagenet.yaml
job.batch/netapp-tensorflow-single-imagenet created
$ kubectl get jobs
NAME                                      COMPLETIONS   DURATION   AGE
netapp-tensorflow-single-imagenet         0/1           24s        24s
```

2. 确认您在步骤 1 中创建的作业正在正确运行。以下示例命令确认已按照作业定义中的指定为此作业创建了一个 POD ，并且此 POD 当前正在其中一个 GPU 工作节点上运行。

```
$ kubectl get pods -o wide
NAME                                                READY   STATUS
RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-single-imagenet-m7x92             1/1     Running     0
3m    10.233.68.61    10.61.218.154   <none>
```

3. 确认您在步骤 1 中创建的作业已成功完成。以下示例命令确认作业已成功完成。

```
$ kubectl get jobs
NAME                                          COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet             1/1           5m42s
10m
$ kubectl get pods
NAME                                                  READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92               0/1     Completed
0          11m
$ kubectl logs netapp-tensorflow-single-imagenet-m7x92
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 702
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 711
Total images/sec = 6530.59125
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 1 -H localhost:1 bash -c 'sync; echo 1 >
/proc/sys/vm/drop_caches'
==========================================
mpirun -allow-run-as-root -np 8 -H localhost:8 -bind-to none -map-by
slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000
--datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_105450_tensorflow_horovod_rdma_resnet50_gpu_8_256_b500_ima
genet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

4. * 可选： * 清理作业项目。以下示例命令显示了在步骤 1 中创建的作业对象的删除情况。

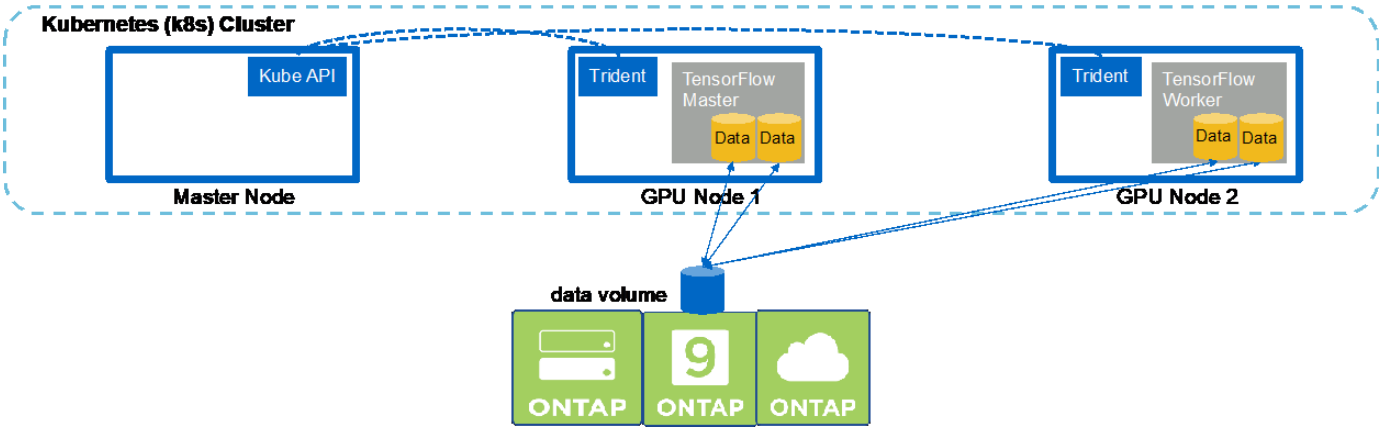
   删除作业对象时， Kubernetes 会自动删除任何关联的 Pod 。

```
$ kubectl get jobs
NAME                                                COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet                   1/1           5m42s
10m
$ kubectl get pods
NAME                                                       READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92                    0/1     Completed
0          11m
$ kubectl delete job netapp-tensorflow-single-imagenet
job.batch "netapp-tensorflow-single-imagenet" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

执行同步分布式 AI 工作负载

要在 Kubernetes 集群中执行同步多节点 AI 和 ML 作业，请在部署跳转主机上执行以下任务。通过此过程，您可以利用存储在 NetApp 卷上的数据，并使用单个工作节点所能提供的 GPU 。有关同步分布式 AI 作业的描述，请参见下图。

> 与异步分布式作业相比，同步分布式作业有助于提高性能和训练准确性。本文档不会讨论同步作业与异步作业的利弊。



1. 以下示例命令显示了创建一名员工参与同步分布式执行本节中示例中在单个节点上执行的同一 TensorFlow 基准测试作业的过程 "执行单节点 AI 工作负载"。在此特定示例中，仅部署一个员工，因为此作业会在两个员工节点上执行。

   此示例员工部署请求八个 GPU ，因此可以在一个 GPU 工作节点上运行，该节点具有八个或更多 GPU 。如果您的 GPU 工作节点具有八个以上的 GPU ，则为了最大限度地提高性能，您可能需要增加此数量，使其等于您的工作节点所具有的 GPU 数量。有关 Kubernetes 部署的详细信息，请参见 "Kubernetes 官方文档"。

在此示例中创建了 Kubernetes 部署，因为此特定容器化员工永远不会自行完成。因此，使用 Kubernetes 作业构造来部署它毫无意义。如果员工的设计或编写是为了自己完成，则可以使用此作业构建来部署员工。

在此示例部署规范中指定的 Pod 的值为 `hostNetwork` 值 `true` 。此值表示 Pod 使用主机工作节点的网络堆栈，而不是 Kubernetes 通常为每个 Pod 创建的虚拟网络堆栈。在这种情况下使用此标注是因为特定工作负载依靠 Open MPI ， NCCL 和 Horovod 以同步分布式方式执行工作负载。因此，它需要访问主机网络堆栈。有关 Open MPI ， NCCL 和 Horovod 的讨论不在本文档的讨论范围之内。是否需要此 `hostNetwork： true` 标注取决于要执行的特定工作负载的要求。有关 `hostNetwork` 字段的详细信息，请参见 "Kubernetes 官方文档"。

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-worker.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: netapp-tensorflow-multi-imagenet-worker
spec:
  replicas: 1
  selector:
    matchLabels:
      app: netapp-tensorflow-multi-imagenet-worker
  template:
    metadata:
      labels:
        app: netapp-tensorflow-multi-imagenet-worker
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["bash", "/netapp/scripts/start-slave-multi.sh",
"22122"]
        resources:
          limits:
            nvidia.com/gpu: 8
```

```
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-worker.yaml
deployment.apps/netapp-tensorflow-multi-imagenet-worker created
$ kubectl get deployments
NAME                                      DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           4s
```

2. 确认您在第 1 步中创建的员工部署已成功启动。以下示例命令确认已为部署创建了一个辅助 POD ，如部署定义所示，并且此 POD 当前正在其中一个 GPU 辅助节点上运行。

```
$ kubectl get pods -o wide
NAME                                                        READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          60s   10.61.218.154   10.61.218.154   <none>
$ kubectl logs netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725
22122
```

3. 为启动，参与并跟踪同步多节点作业执行的主节点创建 Kubernetes 作业。以下示例命令创建一个主节点，用于启动，参与和跟踪在一节中的示例中对单个节点执行的相同 TensorFlow 基准测试作业的同步分布式执行 "执行单节点 AI 工作负载"。

   此示例主作业请求八个 GPU ，因此可以在具有八个或更多 GPU 的单个 GPU 工作节点上运行。如果您的 GPU 工作节点具有八个以上的 GPU ，则为了最大限度地提高性能，您可能需要增加此数量，使其等于您的工作节点所具有的 GPU 数量。

   在本示例作业定义中指定的主 Pod 的值为 `hostNetwork` 值为 `true` ，就像在步骤 1 中为工作 Pod 提供了 `hostNetwork` 值 `true` 一样。有关为何需要此值的详细信息，请参见第 1 步。

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-master.yaml
apiVersion: batch/v1
kind: Job
```

```
metadata:
  name: netapp-tensorflow-multi-imagenet-master
spec:
  backoffLimit: 5
  template:
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--port=22122", "--
num_devices=16", "--dgx_version=dgx1", "--
nodes=10.61.218.152,10.61.218.154"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-master.yaml
job.batch/netapp-tensorflow-multi-imagenet-master created
$ kubectl get jobs
```

```
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   0/1           25s        25s
```

4. 确认您在步骤 3 中创建的主作业正在正确运行。以下示例命令确认已为作业创建了一个主 Pod ，如作业定义所示，并且此 Pod 当前正在其中一个 GPU 工作节点上运行。您还应看到，您最初在步骤 1 中看到的辅助 POD 仍在运行，并且主节点和辅助节点正在不同的节点上运行。

```
$ kubectl get pods -o wide
NAME                                                          READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-master-ppwwj                 1/1
Running   0          45s   10.61.218.152   10.61.218.152   <none>
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          26m   10.61.218.154   10.61.218.154   <none>
```

5. 确认您在步骤 3 中创建的主作业已成功完成。以下示例命令确认作业已成功完成。

```
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      9m18s
$ kubectl get pods
NAME                                                          READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj                 0/1
Completed   0          9m38s
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          35m
$ kubectl logs netapp-tensorflow-multi-imagenet-master-ppwwj
[10.61.218.152:00008] WARNING: local probe returned unhandled
shell:unknown assuming bash
rm: cannot remove '/lib': Is a directory
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
Total images/sec = 12881.33875
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 2 -H 10.61.218.152:1,10.61.218.154:1 -mca
pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" bash -c 'sync; echo 1 >
```

```
/proc/sys/vm/drop_caches'
========================================
mpirun -allow-run-as-root -np 16 -H 10.61.218.152:8,10.61.218.154:8
-bind-to none -map-by slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH
-mca pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -x
NCCL_IB_HCA=mlx5 -x NCCL_NET_GDR_READ=1 -x NCCL_IB_SL=3 -x
NCCL_IB_GID_INDEX=3 -x
NCCL_SOCKET_IFNAME=enp5s0.3091,enp12s0.3092,enp132s0.3093,enp139s0.3094
-x NCCL_IB_CUDA_SUPPORT=1 -mca orte_base_help_aggregate 0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000 --
datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_161609_tensorflow_horovod_rdma_resnet50_gpu_16_256_b500_im
agenet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

6. 如果您不再需要此员工部署，请将其删除。以下示例命令显示了删除在步骤 1 中创建的工作部署对象的过程。

   删除 worker 部署对象时， Kubernetes 会自动删除任何关联的 worker Pod 。

```
$ kubectl get deployments
NAME                                        DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           43m
$ kubectl get pods
NAME                                                        READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj              0/1
Completed   0          17m
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          43m
$ kubectl delete deployment netapp-tensorflow-multi-imagenet-worker
deployment.extensions "netapp-tensorflow-multi-imagenet-worker" deleted
$ kubectl get deployments
No resources found.
$ kubectl get pods
NAME                                              READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
18m
```

7. * 可选： * 清理主作业项目。以下示例命令显示了删除在步骤 3 中创建的主作业对象的过程。

   删除主作业对象时， Kubernetes 会自动删除任何关联的主 Pod 。

```
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      19m
$ kubectl get pods
NAME                                              READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
19m
$ kubectl delete job netapp-tensorflow-multi-imagenet-master
job.batch "netapp-tensorflow-multi-imagenet-master" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

性能测试

在创建此解决方案时，我们执行了简单的性能比较。我们使用 Kubernetes 执行了多个标准 NetApp AI 基准测试作业，并将基准测试结果与使用简单 Docker run 命令执行的执行情况进行了比较。我们没有发现任何明显的性能差异。因此，我们得出的结论是，使用 Kubernetes 编排容器化 AI 培训作业不会对性能产生负面影响。有关性能比较结果，请参见下表。

基准测试	数据集	**Docker** 运行（映像 / 秒）	**Kubernetes** （图像 / 秒）
单节点 TensorFlow	合成数据	6,667.2475	6,661.93125
单节点 TensorFlow	ImageNet	6,570.2025	6,530.59125
同步分布式双节点 TensorFlow	合成数据	13,213.70625	13,218.288125
同步分布式双节点 TensorFlow	ImageNet	12,941.69125	12,881.33875

结论

各行各业各种规模的企业和组织都在转向人工智能（AI），机器学习（ML）和深度学习（DL），以解决实际问题，提供创新产品和服务，并在竞争日益激烈的市场中占据优势。随着企业越来越多地使用 AI， ML 和 DL，他们面临着许多挑战，包括工作负载可扩展性和数据可用性。可以通过使用 NetApp AI 控制平台解决方案来应对这些挑战。

通过此解决方案，您可以快速克隆数据命名空间。此外，您还可以定义和实施 AI， ML 和 DL 培训工作流，这些工作流可近乎即时地创建数据和模型基线，以实现可追溯性和版本控制。使用此解决方案，您可以跟踪每个模型训练返回到模型经过训练和 / 或验证的确切数据集。最后，借助此解决方案，您可以快速配置 Jupyter 笔记本电脑工作空间，以访问海量数据集。

由于此解决方案面向数据科学家和数据工程师，因此只需极少的 NetApp 或 NetApp ONTAP 专业知识即可。借助此解决方案，可以使用简单熟悉的工具和界面来执行数据管理功能。此外，此解决方案还利用完全开源和免费的组件。因此，如果您的环境中已有 NetApp 存储，则可以立即实施此解决方案。如果您要测试此解决方案的驱动器，但尚未安装 NetApp 存储，请访问 "cloud.netapp.com"，您可以随时使用基于云的 NetApp 存储解决方案启动和运行。

版权信息

版权所有 © 2024 NetApp, Inc.。保留所有权利。中国印刷。未经版权所有者事先书面许可，本文档中受版权保护的任何部分不得以任何形式或通过任何手段（图片、电子或机械方式，包括影印、录音、录像或存储在电子检索系统中）进行复制。

从受版权保护的 NetApp 资料派生的软件受以下许可和免责声明的约束：

本软件由 NetApp 按“原样”提供，不含任何明示或暗示担保，包括但不限于适销性以及针对特定用途的适用性的隐含担保，特此声明不承担任何责任。在任何情况下，对于因使用本软件而以任何方式造成的任何直接性、间接性、偶然性、特殊性、惩罚性或后果性损失（包括但不限于购买替代商品或服务；使用、数据或利润方面的损失；或者业务中断），无论原因如何以及基于何种责任理论，无论出于合同、严格责任或侵权行为（包括疏忽或其他行为），NetApp 均不承担责任，即使已被告知存在上述损失的可能性。

NetApp 保留在不另行通知的情况下随时对本文档所述的任何产品进行更改的权利。除非 NetApp 以书面形式明确同意，否则 NetApp 不承担因使用本文档所述产品而产生的任何责任或义务。使用或购买本产品不表示获得 NetApp 的任何专利权、商标权或任何其他知识产权许可。

本手册中描述的产品可能受一项或多项美国专利、外国专利或正在申请的专利的保护。

有限权利说明：政府使用、复制或公开本文档受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中“技术数据权利 — 非商用”条款第 (b)(3) 条规定的限制条件的约束。

本文档中所含数据与商业产品和/或商业服务（定义见 FAR 2.101）相关，属于 NetApp, Inc. 的专有信息。根据本协议提供的所有 NetApp 技术数据和计算机软件具有商业性质，并完全由私人出资开发。 美国政府对这些数据的使用权具有非排他性、全球性、受限且不可撤销的许可，该许可既不可转让，也不可再许可，但仅限在与交付数据所依据的美国政府合同有关且受合同支持的情况下使用。除本文档规定的情形外，未经 NetApp, Inc. 事先书面批准，不得使用、披露、复制、修改、操作或显示这些数据。美国政府对国防部的授权仅限于 DFARS 的第 252.227-7015(b)（2014 年 2 月）条款中明确的权利。

商标信息

NetApp、NetApp 标识和 http://www.netapp.com/TM 上所列的商标是 NetApp, Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。